Introduction

- **What is Git?**

Git is an **open-source distributed version control system**. It is designed to handle minor to major projects with high speed and efficiency. It is developed to co-ordinate the work among the developers. The version control allows us to track and work together with our team members at the same workspace.

Git is easy to learn, and has fast performance. It is superior to other SCM tools like Subversion, CVS, Perforce, and ClearCase.

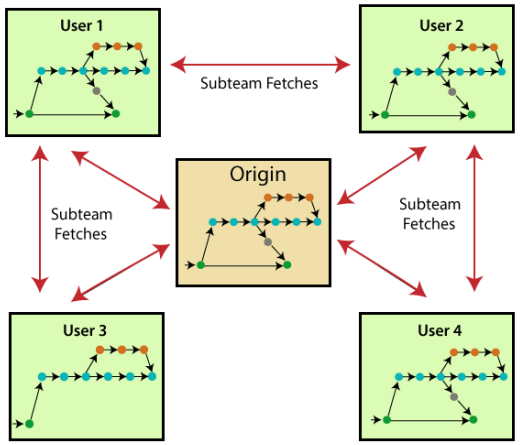
Features of Git

1. Open Source
2. Scalable
3. Distributed
4. Security
5. Speed
6. Supports non-linear development
7. Branching and Merging
8. Data Assurance
9. Staging Area
10. Maintain the clean history



- **Open Source**
  Git is an open-source tool. It is released under the **GPL** (General Public License) license.
- **Scalable**
  Git is scalable, which means when the number of users increases, the Git can easily handle such situations.
- **Distributed:**

One of Git's great features is that it is **distributed**. Distributed means that instead of switching the project to another machine, we can create a "clone" of the entire repository. Also, instead of just having one central repository that you send changes to, every user has their own repository that contains the entire commit history of the project. We do not need to connect to the remote repository; the change is just stored on our local repository. If necessary, we can push these changes to a remote repository.



- **Security**
  Git is secure. It uses the SHA1 (Secure Hash Function) to name and identify objects within its repository. Files and commits are checked and retrieved by its checksum at the time of checkout. It stores its history in such a way that the ID of particular commits depends upon the complete development history leading up to that commit. Once it is published, one cannot make changes to its old version.
- **Speed**
  Git is very fast, so it can complete all the tasks in a while. Most of the git operations are done on the local repository, so it provides a **huge speed**. Also, a centralized version control system continually communicates with a server somewhere.
  Performance tests conducted by Mozilla showed that it was **extremely fast compared to other VCSs**. Fetching version history from a locally stored repository is much faster than fetching it from the remote server. The **core part of Git** is **written in C**, which **ignores** runtime overheads associated with other high-level languages.
  Git was developed to work on the Linux kernel; therefore, it is **capable** enough to **handle large repositories** effectively. From the beginning, **speed** and **performance** have been Git's primary goals.
- **Supports non-linear development**
  Git supports seamless branching and merging, which helps in visualizing and navigating a non-linear development. A branch in Git represents a single commit. We can construct the full branch structure with the help of its parental commit.

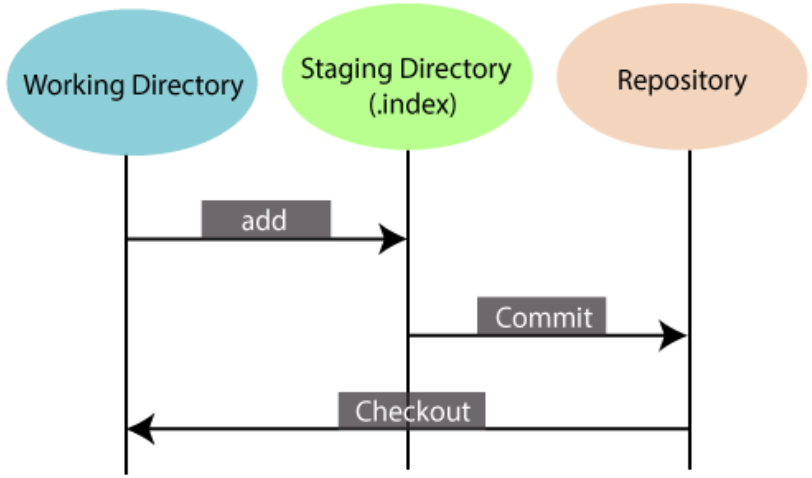
- **Branching and merging**

Branching and merging are the **great feature**s of Git, which makes it different from the other SCM tools. Git allows the **creation of multiple branches** without affecting each other. We can perform tasks like **creation**, **deletion**, and **merging** on branches, and these tasks take a few seconds only. Below are some features that can be achieved by branching:

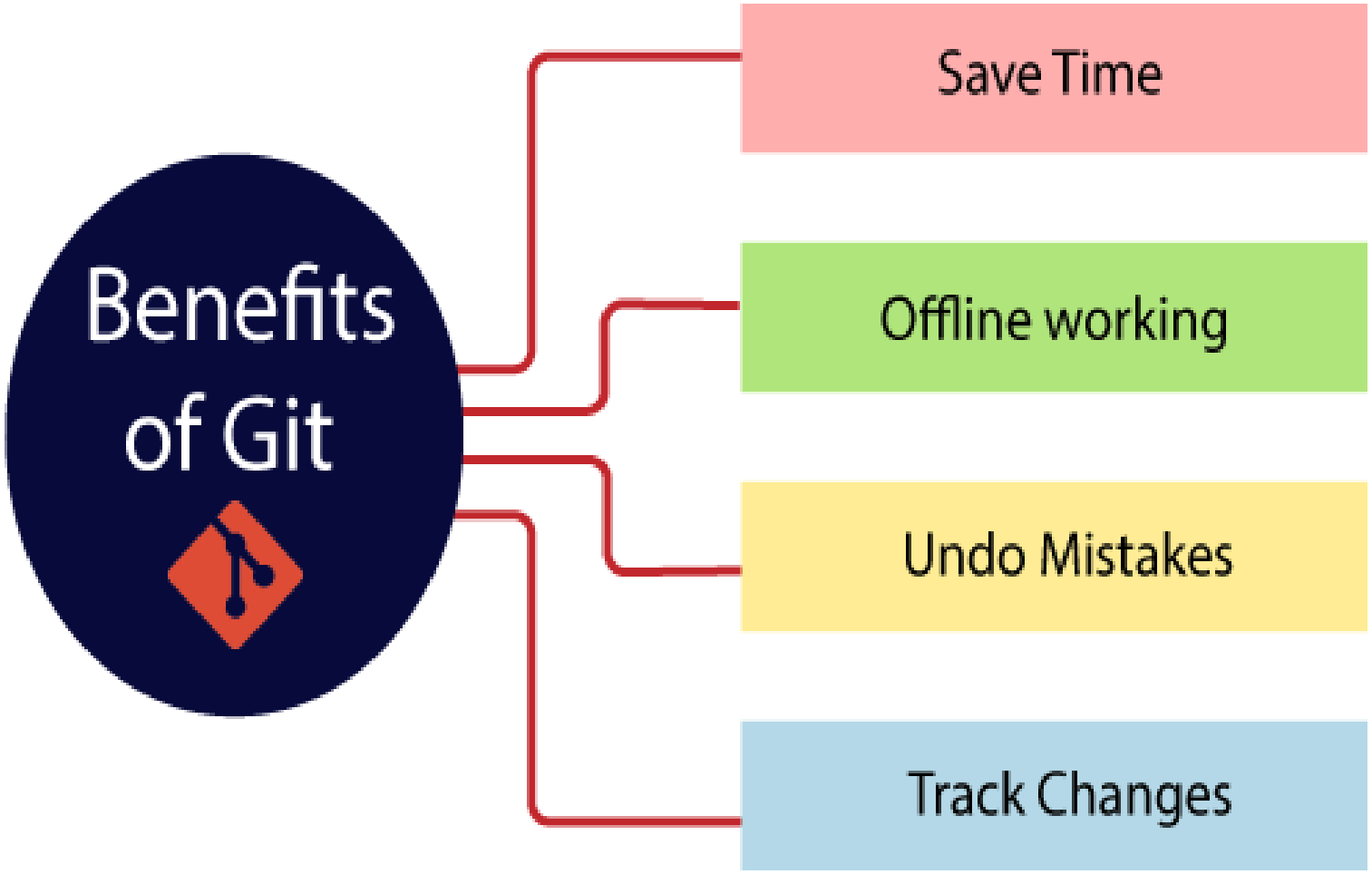
- We can **create a separate branch** for a new module of the project, commit and delete it whenever we want.
- We can have a **production branch**, which always has what goes into production and can be merged for testing in the test branch.
- We can create a **demo branch** for the experiment and check if it is working. We can also remove it if needed.
- The core benefit of branching is if we want to push something to a remote repository, we do not have to push all of our branches. We can select a few of our branches, or all of them together.

- **Staging Area**

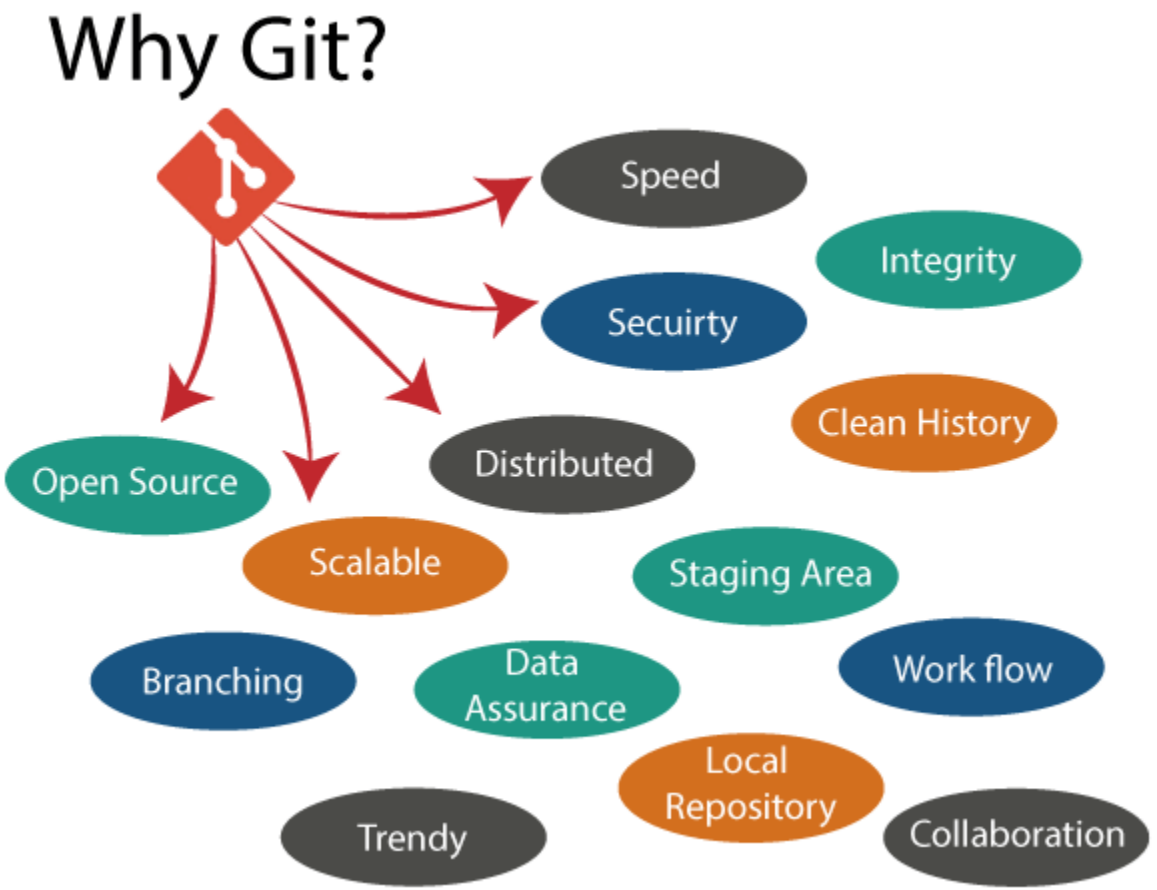
The **Staging area** is also a **unique functionality** of Git. It can be considered as a **preview of our next commit**, moreover, an **intermediate area** where commits can be formatted and reviewed before completion. When you make a commit, Git takes changes that are in the staging area and make them as a new commit. We are allowed to add and remove changes from the staging area. The staging area can be considered as a place where Git stores the changes.
Although, Git doesn't have a dedicated staging directory where it can store some objects representing file changes (blobs). Instead of this, it uses a file called index. Another feature of Git that makes it apart from other SCM tools is that **it is possible to quickly stage some of our files and commit them without committing other modified files in our working directory.**



Benefits of Git



Why Git?


Why Git?
Speed
Integrity
Securirty
Clean History
Distributed
Open Source
Scalable
Staging Area
Branching
Data Assurance
Work flow
Local Repository
Trendy
Collaboration

- Git Integrity
  Git is developed to ensure the security and integrity of content being version controlled. It uses checksum during transit or tampering with the file system to confirm that information is not lost. Internally it creates a checksum value from the contents of the file and then verifies it when transmitting or storing data.
- Trendy Version Control System
  Git is the most widely used version control system. It has maximum projects among all the version control systems. Due to its amazing workflow and features, it is a preferred choice of developers.

- Everything is Local
  Almost All operations of Git can be performed locally; this is a significant reason for the use of Git. We will not have to ensure internet connectivity.
- Collaborate to Public Projects
  There are many public projects available on the GitHub. We can collaborate on those projects and show our creativity to the world. Many developers are collaborating on public projects. The collaboration allows us to stand with experienced developers and learn a lot from them; thus, it takes our programming skills to the next level.
- Impress Recruiters
  We can impress recruiters by mentioning the Git and GitHub on our resume. Send your GitHub profile link to the HR of the organization you want to join. Show your skills and influence them through your work. It increases the chances of getting hired.

What is GitHub?

GitHub is a Git repository hosting service. GitHub also facilitates with many of its features, such as access control and collaboration. It provides a Web-based graphical interface.

GitHub is an American company. It hosts source code of your project in the form of different programming languages and keeps track of the various changes made by programmers.

It offers both **distributed version control and source code management (SCM)** functionality of Git. It also facilitates with some collaboration features such as bug tracking, feature requests, task management for every project.

Some of its significant features are as follows.



The Need for Source Code

- Source code is the human-readable form of a program, written in programming languages like Python, Java, C++, etc.
- It acts as the blueprint for software applications.
- Maintaining and managing source code effectively is crucial for collaboration, debugging, and software evolution.
- Example - Imagine building a weather forecasting app. Developers write the code (source code) in a repository.
- Each developer contributes their part—some handle the user interface, while others handle data fetching from APIs.

Git Version Control System

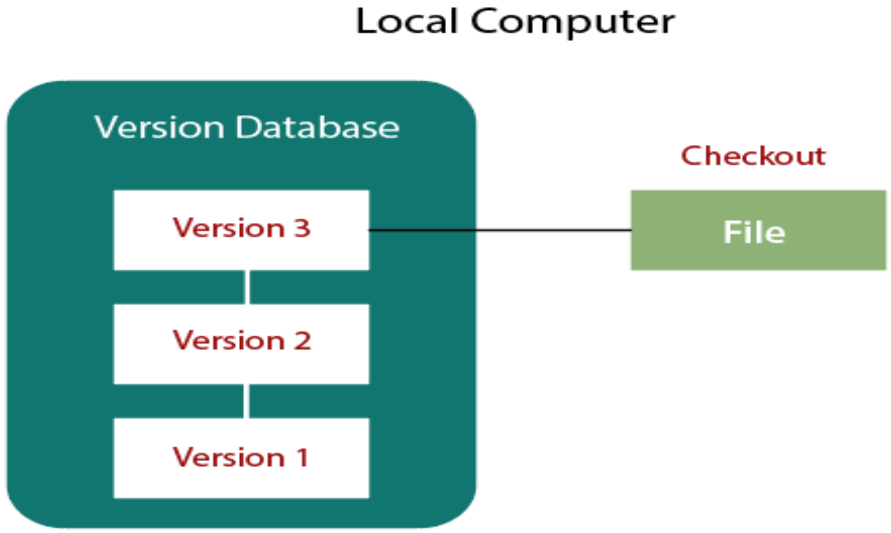
- A version control system is a software that tracks changes to a file or set of files over time so that you can recall specific versions later. It also allows you to work together with other programmers.
- The version control system is a collection of software tools that help a team to manage changes in a source code. It uses a special kind of database to keep track of every modification to the code.
- Developers can compare earlier versions of the code with an older version to fix the mistakes.

Benefits of the Version Control System

The Version Control System is very helpful and beneficial in software development; developing software without using version control is unsafe. It provides backups for uncertainty. Version control systems offer a speedy interface to developers. It also allows software teams to preserve efficiency and agility according to the team scales to include more developers.

Types of Version Control System

- Localized version Control System
- Centralized version control systems
- Distributed version control systems



Localized Version Control Systems

- The localized version control method is a common approach because of its simplicity. But this approach leads to a higher chance of error. In this approach, you may forget which directory you're in and accidentally write to the wrong file or copy over files you don't want to.
- The Version Control System is very helpful and beneficial in software development; developing software without using version control is unsafe. It provides backups for uncertainty. Version control systems offer a speedy interface to developers. It also allows software teams to preserve efficiency and agility according to the team scales to include more developers.

Benefits of VCS

Some key benefits of having a version control system are as follows.

- Complete change history of the file
- Simultaneously working
- Branching and merging
- Traceability

Centralized Version Control System

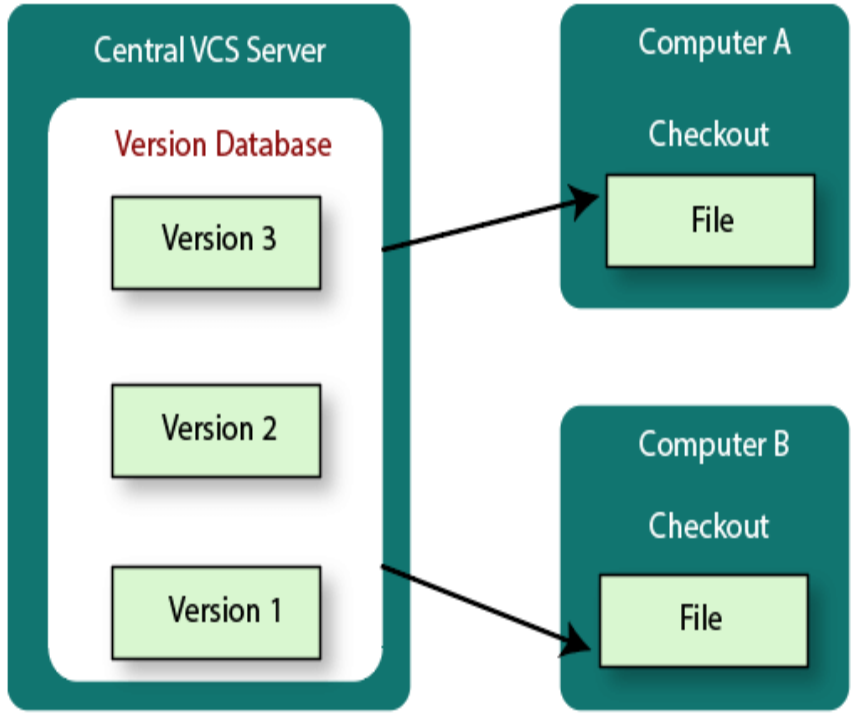
The developers needed to collaborate with other developers on other systems. The localized version control system failed in this case. To deal with this problem, Centralized Version Control Systems were developed.

These systems have a single server that contains the versioned files, and some clients to check out files from a central place.

Centralized version control systems have many benefits, especially over local VCSs.

- Everyone on the system has information about the work what others are doing on the project.
- Administrators have control over other developers.
- It is easier to deal with a centralized version control system than a localized version control system.
- A local version control system facilitates with a server software component which stores and manages the different versions of the files.

It also has the same drawback as in local version control system that it also has a single point of failure.



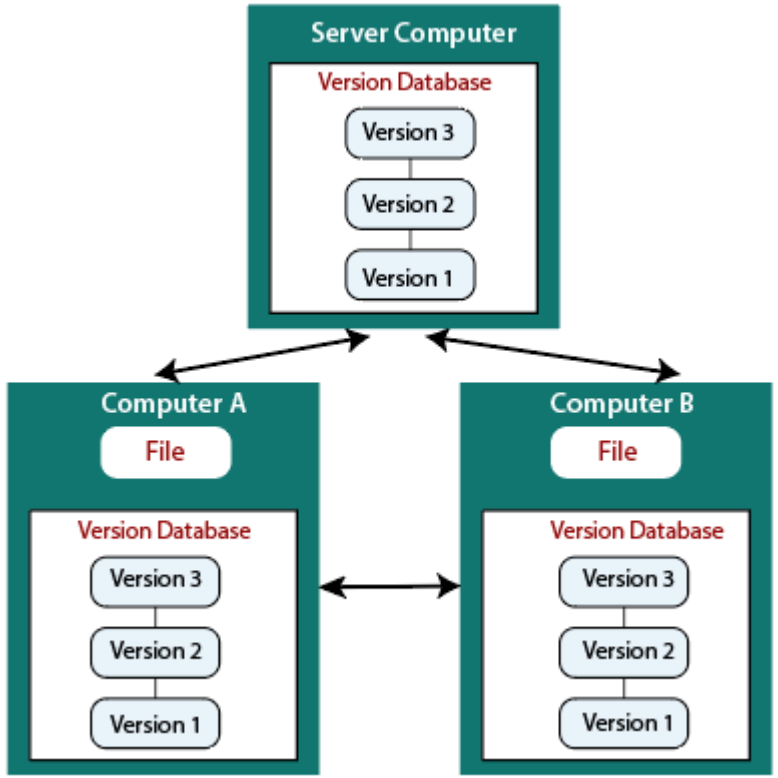
Distributed Version Control System

Centralized Version Control System uses a central server to store all the database and team collaboration. But due to single point failure, which means the failure of the central server, developers do not prefer it. Next, the Distributed Version Control System is developed.

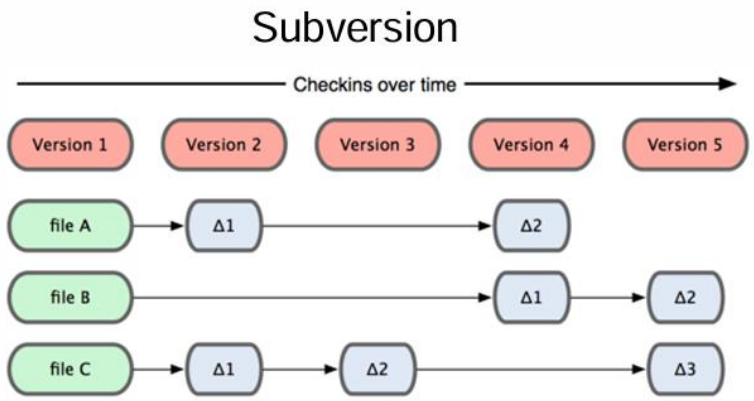
In a Distributed Version Control System (such as Git, Mercurial, Bazaar or Darcs), the user has a local copy of a repository. So, the clients don't just check out the latest snapshot of the files even they can fully mirror the repository. The local repository contains all the files and metadata present in the main repository.

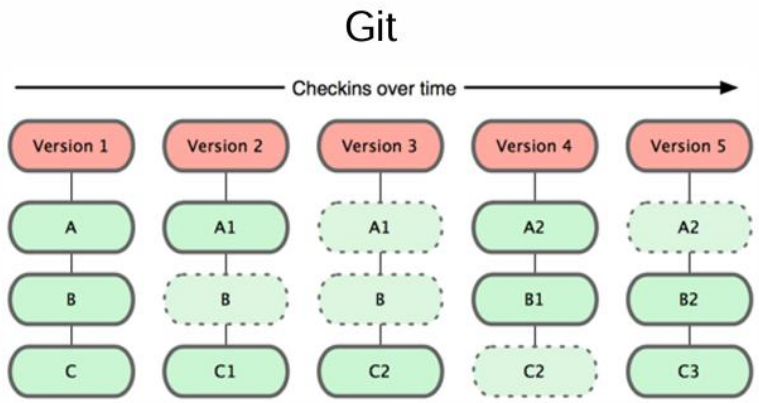
DVCS allows automatic management branching and merging. It speeds up of most operations except pushing and pulling. DVCS enhances the ability to work offline and does not rely on a single location for backups. If any server stops and other systems were collaborating via it, then any of the client repositories could be restored by that server. Every checkout is a full backup of all the data.

These systems do not necessarily depend on a central server to store all the versions of a project file.



What is Git - Snapshots, Not Differences





Git Has Integrity - Git uses checksums

- In Subversion each modification to the central repo incremented the version # of the overall repo.
- How will this numbering scheme work when each user has their own copy of the repo, and commits changes to their local copy of the repo before pushing to the central server?????
- Instead, Git generates a unique SHA-1 hash – 40 character string of hex digits, for every commit. Refer to commits by this ID rather than a version number.

  Often we only see the first 7 characters:

  1677b2d Edited first line of readme

  258efa7 Added line to readme

  0e52da7 Initial commit

Git Has Integrity - Git uses checksums

- In Git, versioning works fundamentally differently from Subversion (SVN) due to its **distributed** nature. Here's how Git handles versioning:
- **No Global Version Number**:
  - In Subversion, every commit increases a global repository-wide version number.
  - In Git, each commit gets a unique **SHA-1 hash** instead of a sequential version number.
- **Local Commits First**:
  - Users commit changes to their **local** repository first.
  - These commits exist **only in their local copy** until they push them to the central repository (e.g., GitHub, GitLab).
  - Since multiple users can work in parallel, their local commits may have different histories before merging.

Git Has Integrity - Git uses checksums

SHA-1 Hash for Versioning:

- Each commit is identified by a **40-character SHA-1 hash** (e.g., `1677b2d5e...`).
- This hash is based on the commit's contents, timestamp, and parent commits.
- The first **7 characters** of the hash are usually shown in logs (`git log --oneline`).

Commit History Merging:

- When users push their commits to the central repository, Git **merges** these commits into the main branch.
- If multiple users push simultaneously, Git may require a merge (`git pull --rebase` or `git merge`).
- The commit history remains unique for each change.

Git Has Integrity - Git uses checksums

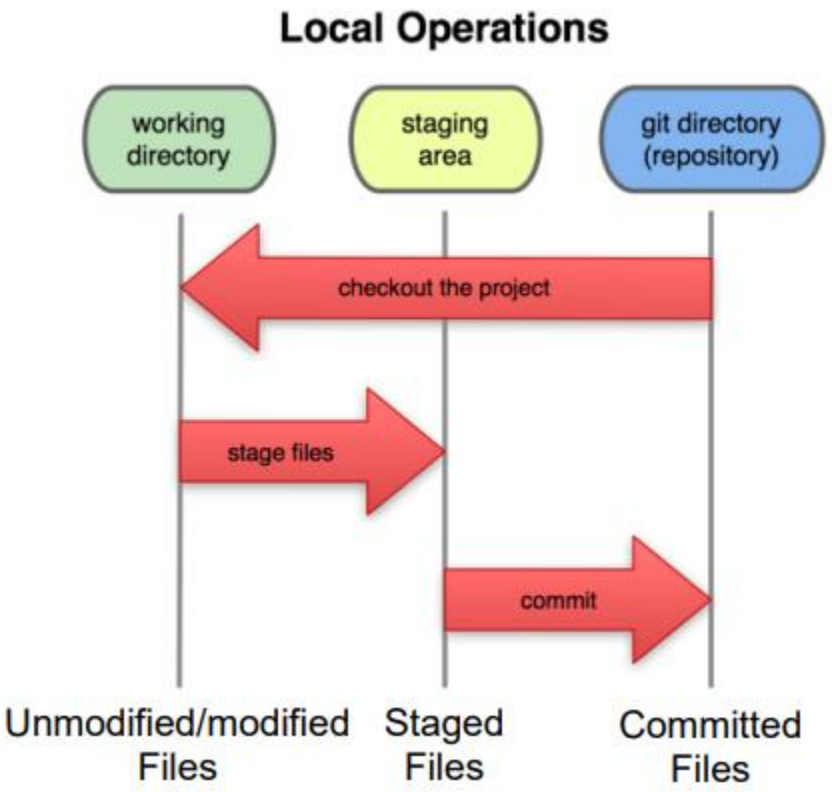
- $ git log --oneline
- 1677b2d Edited first line of readme
- 258efa7 Added line to readme
- 0e52da7 Initial commit

Here, each commit has a unique SHA-1 identifier instead of a sequential number.

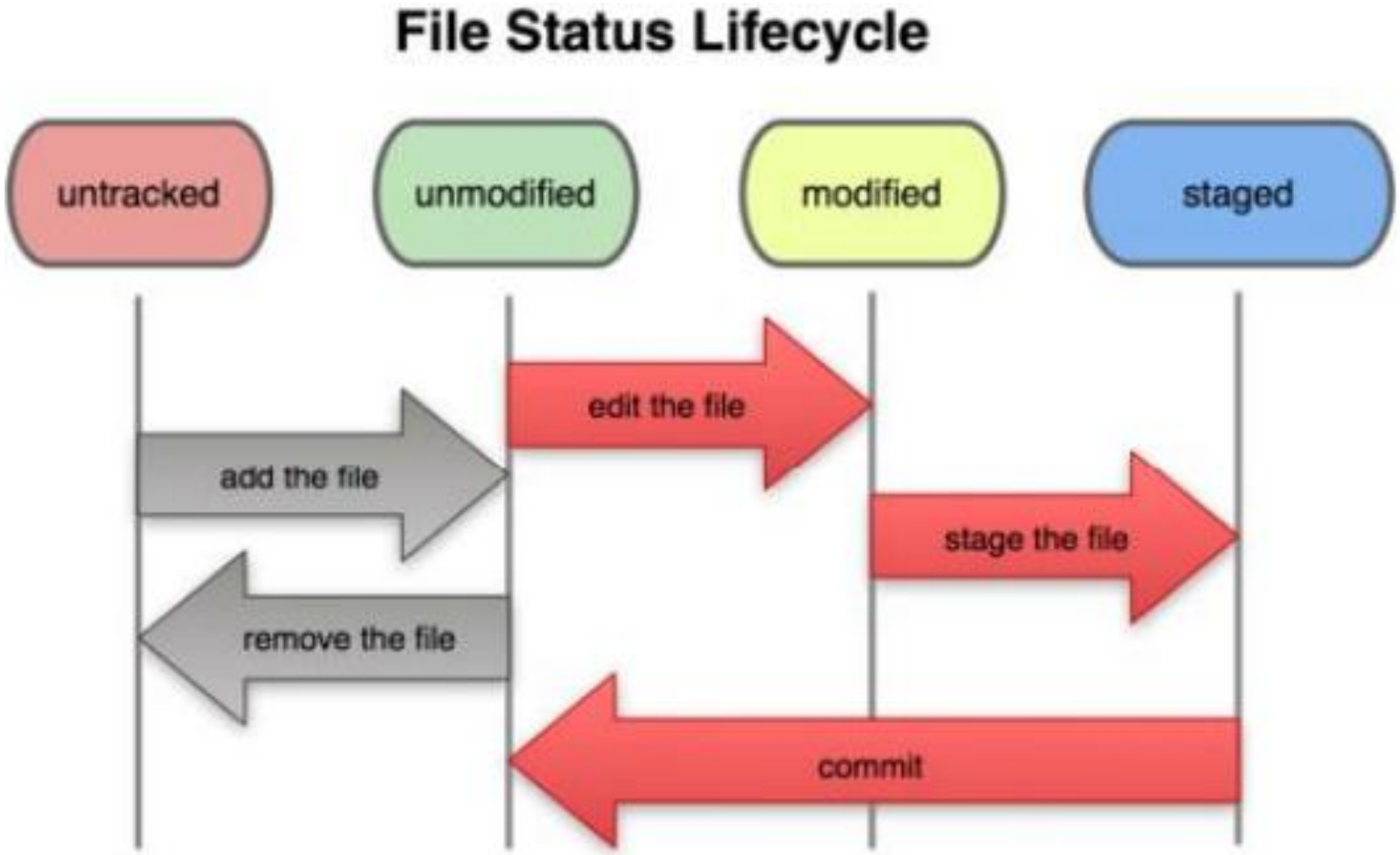
Why This is Better than SVN:

- ☑ Supports **distributed workflows** (work offline, commit locally).
- ☑ No risk of **conflicting version numbers** when multiple developers work in parallel.
- ☑ Every commit is **cryptographically secure** (ensures integrity).

A Local Git project has three areas



Git file lifecycle



Basic Workflow

Basic Git workflow:

1. **Modify** files in your working directory.
2. **Stage** files, adding snapshots of them to your staging area.
3. Do a **commit**, which takes the files as they are in the staging area and stores that snapshot permanently to your Git directory.

- Notes:
  - If a particular version of a file is in the **git directory**, it's considered **committed**.
  - If it's modified but has been added to the **staging area**, it is **staged**.
  - If it was **changed** since it was checked out but has not been staged, it is **modified**.

Aside: So what is github?

- GitHub.com is a site for online storage of Git repositories.
- Many open source projects use it, such as the Linux kernel.
- You can get free space for open source projects or you can pay for private projects.

Question: Do I have to use github to use Git?

Answer: No!

- you can use Git completely locally for your own purposes, or
- you or someone else could set up a server to share files, or
- you could share a repo with users on the same file system, such as we did for homework 9 (as long everyone has the needed file permissions).

Get ready to use Git!

1. Set the name and email for Git to use when you commit:

```
$ git config --global user.name "Bugs Bunny"
$ git config --global user.email bugs@gmail.com
```

- You can call **`git config –list`** to verify these are set.
- These will be set globally for all Git projects you work with.
- You can also set variables on a project-only basis by not using the **`--global`** flag.
- You can also set the editor that is used for writing commit messages:
  $ git config --global core.editor emacs (it is vim by default)

Create a local copy of a repo

2. Two common scenarios: (only do one of these)

a) To **clone an already existing repo** to your current directory:

```
$ git clone <url> [local dir name]
```

This will create a directory named *local dir name*, containing a working copy of the files from the repo, and a **.git** directory (used to hold the staging area and your actual repo)

b) To **create a Git repo** in your current directory:

```
$ git init
```

This will create a **.git** directory in your current directory.

Then you can commit files in that directory into the repo:

```
$ git add file1.java
$ git commit –m “initial project version”
```

Git commands

command	description
`git clone` ***`url [dir]`***	copy a git repository so you can add to it
`git add` ***`files`***	adds file contents to the staging area
`git commit`	records a snapshot of the staging area
`git status`	view the status of your files in the working directory and staging area
`git diff`	shows diff of what is staged and what is modified but unstaged
`git help` ***`[command]`***	get help info about a particular command
`git pull`	fetch from a remote repo and try to merge into the current branch
`git push`	push your new branches and data to a remote repository
others: `init, reset, branch, checkout, merge, log, tag`	

Committing files

- The first time we ask a file to be tracked, *and* **every time before we commit a file** we must add it to the staging area:

```
$ git add README.txt hello.java
```

This takes a snapshot of these files at this point in time and adds it to the staging area.

- To move staged changes into the repo we commit:

```
$ git commit –m “Fixing bug #22”
```

Note: To unstage a change on a file before you have committed it:

```
$ git reset HEAD  --  filename
```

Note: To unmodify a modified file:

```
$ git checkout  --  filename
```

Note: These commands are just acting on **your local version of repo**.

Status and Diff

- To view the **status** of your files in the working directory and staging area:

```
$ git status          or
$ git status -s
```

(`-s` shows a short one line version similar to svn)

- To see what is modified but unstaged:

```
$ git diff
```

- To see staged changes:

```
$ git diff --cached
```



After editing a file…

```
[rea@attu1 superstar]$ emacs rea.txt
[rea@attu1 superstar]$ git status
# On branch master
# Changes not staged for commit:
#   (use "git add <file>..." to update what will be committed)
#   (use "git checkout -- <file>..." to discard changes in working directory)
#
#       modified:   rea.txt
#
no changes added to commit (use "git add" and/or "git commit -a")
[rea@attu1 superstar]$ git status -s
 M rea.txt                          ← Note: M is in second column = "working  tree"
[rea@attu1 superstar]$ git diff     ← Shows modifications that have not been staged.
diff --git a/rea.txt b/rea.txt
index 66b293d..90b65fd 100644
--- a/rea.txt
+++ b/rea.txt
@@ -1,2 +1,4 @@
 Here is rea's file.
+
+One new line added.
[rea@attu1 superstar]$ git diff --cached   ← Shows nothing, no modifications have been staged yet.
[rea@attu1 superstar]$
```

After adding file to staging area…

```
[rea@attu1 superstar]$ git add rea.txt
[rea@attu1 superstar]$ git status
# On branch master
# Changes to be committed:
#   (use "git reset HEAD <file>..." to unstage)
#
#       modified:   rea.txt
#
[rea@attu1 superstar]$ git status -s
M  rea.txt                                  ← Note: M is in first column = "staging area"
[rea@attu1 superstar]$ git diff             ← Note: Shows nothing, no modifications that have not been staged.
[rea@attu1 superstar]$ git diff --cached    ← Note: Shows staged modifications.
diff --git a/rea.txt b/rea.txt
index 66b293d..90b65fd 100644
--- a/rea.txt
+++ b/rea.txt
@@ -1,2 +1,4 @@
 Here is rea's file.
+
+One new line added.
```

Viewing logs

To see a log of all changes in your local repo:

- `$ git log` or
- `$ git log --oneline` (to show a shorter version)

```
1677b2d Edited first line of readme
258efa7 Added line to readme
0e52da7 Initial commit
```

- `git log -5` (to show only the 5 most recent updates, etc.)

Note: changes will be listed by commitID #, (SHA-1 hash)

Note: changes made to the remote repo before the last time you cloned/pulled from it will also be included here

Pulling and Pushing

Good practice:

1. **Add** and **Commit** your changes to your local repo
2. **Pull** from remote repo to get most recent changes (fix conflicts if necessary, add and commit them to your local repo)
3. **Push** your changes to the remote repo

To fetch the most recent updates from the remote repo into your local repo, and put them into your working directory:

```
$ git pull origin master
```

To push your changes from your local repo to the remote repo:

```
$ git push origin master
```

Notes: **`origin`** = an alias for the URL you cloned from

`master` = the remote branch you are pulling from/pushing to,
(the local branch you are pulling to/pushing from is your current branch)

Note: On attu you will get a Gtk-warning, you can ignore this.

Branching

To create a branch called experimental:

- `$ git branch experimental`

To list all branches: (* shows which one you are currently on)

- `$ git branch`

To switch to the experimental branch:

- `$ git checkout experimental`

Later on, changes between the two branches differ, to merge changes from experimental into the master:

- `$ git checkout master`
- `$ git merge experimental`

Note: `git log --graph` can be useful for showing branches.

Note: These branches are in *your local repo*!

Do This:

1. $ git config --global user.name “Your Name”
2. $ git config --global user.email youremail@whatever.com
3. $ git clone https://github.com/rea2000/santalist.git

Then try:

1. $ git log, $ git log --oneline
2. Create a file named *userID*.txt (e.g. rea.txt)
3. $ git status, $ git status -s
4. Add the file: $ git add *userID*.txt
5. $ git status, $ git status -s
6. Commit the file to your local repo:
   $ git commit -m “added rea.txt file”
7. $ git status, $ git status -s, $ git log --oneline

*WAIT, DO NOT GO ON TO THE NEXT STEPS UNTIL YOU ARE TOLD TO!!

1. Pull from remote repo: $git pull origin master
2. Push to remote repo: $git push origin master

Git Branching

- Nearly every VCS has some form of branching support.
- Branching means you diverge from the main line of development and continue to do work without messing with that main line.
- In many VCS tools, this is a somewhat expensive process, often requiring you to create a new copy of your source code directory, which can take a long time for large projects.

Branches in a Nutshell

- **Understanding Git Branching: How Git Stores Data**
- Git stores data as a series of **snapshots**, not changesets or differences.
- Each commit creates a **commit object** that includes:
  - A pointer to the **snapshot** of staged content.
  - Author's **name and email address**.
  - The **commit message**.
  - Pointers to previous **commit(s)** (parents):
    - **Zero parents** → Initial commit.
    - **One parent** → Normal commit.
    - **Multiple parents** → Merge commit (from multiple branches).

To visualize this, let's assume that you have a directory containing three files, and you stage them all and commit.

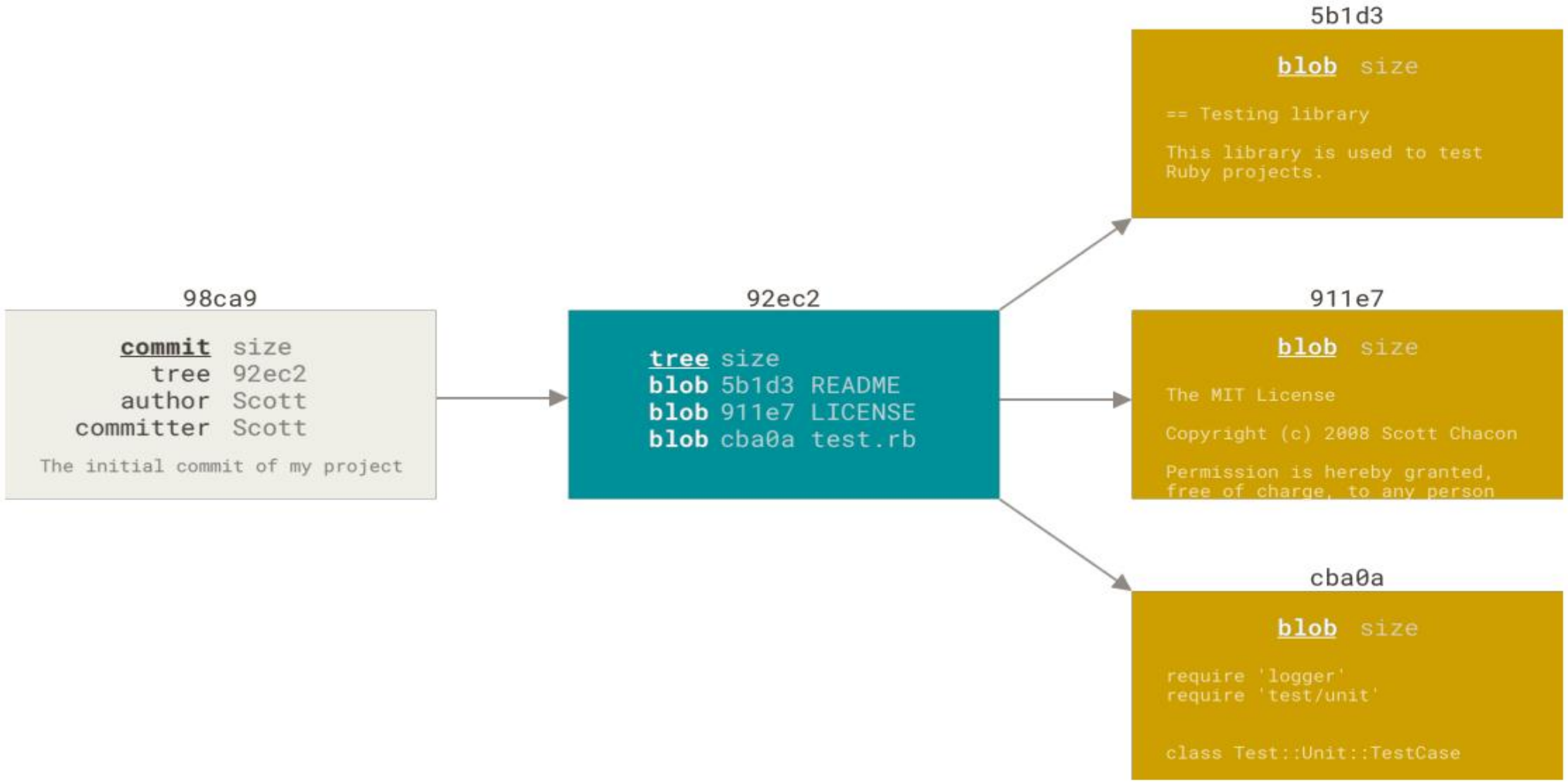
```
$ git add README test.rb LICENSE
$ git commit -m 'Initial commit'
```

Visualizing Git Data Storage

- **Staging Files**
  - Computes a **SHA-1 checksum** for each file.
  - Stores the **file versions as blobs** in the Git repository.
  - Adds the **checksum** to the staging area.
  - Example command:

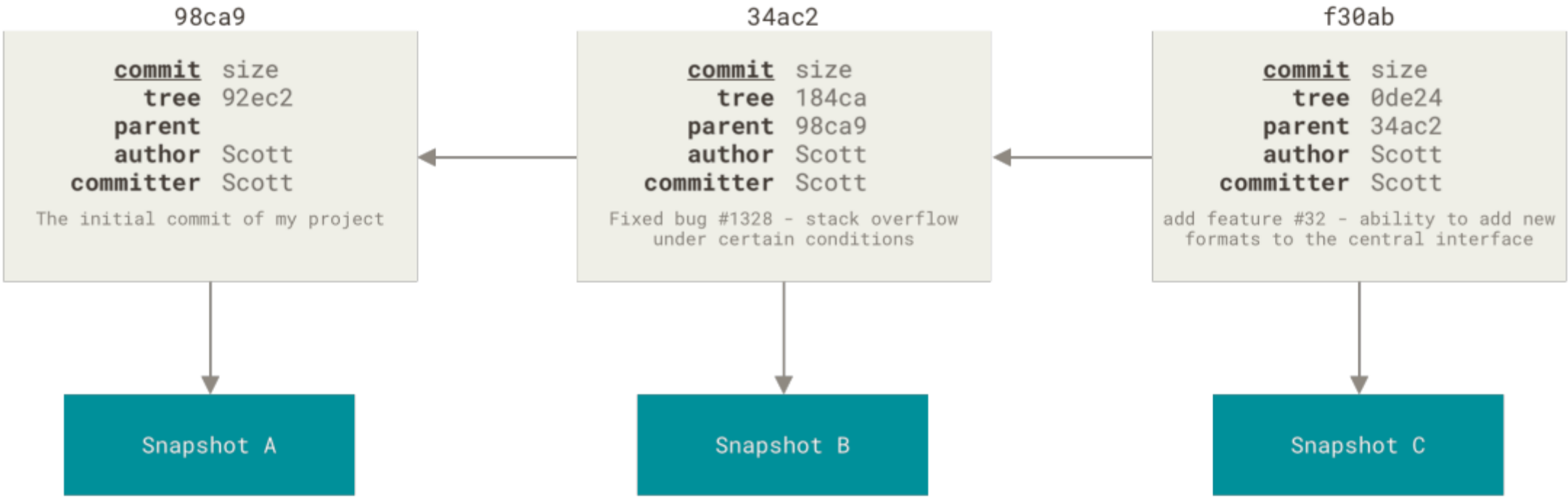
  - `$ git add README test.rb LICENSE`
  - `$ git commit -m 'Initial commit'`
  -
- **Creating a Commit**
  - Git **checksums the root directory** and stores it as a **tree object**.
  - Generates a **commit object** with:
    - **Metadata** (author, message, timestamp).
    - A **pointer** to the root tree for snapshot recreation.
- **Git Repository Structure After Commit**
  - Contains **five objects**:
    - **Three blobs** (representing file contents).
    - **One tree** (maps filenames to blobs).
    - **One commit** (points to the root tree & stores metadata).

Visualizing Git Data Storage



Commits and their parents

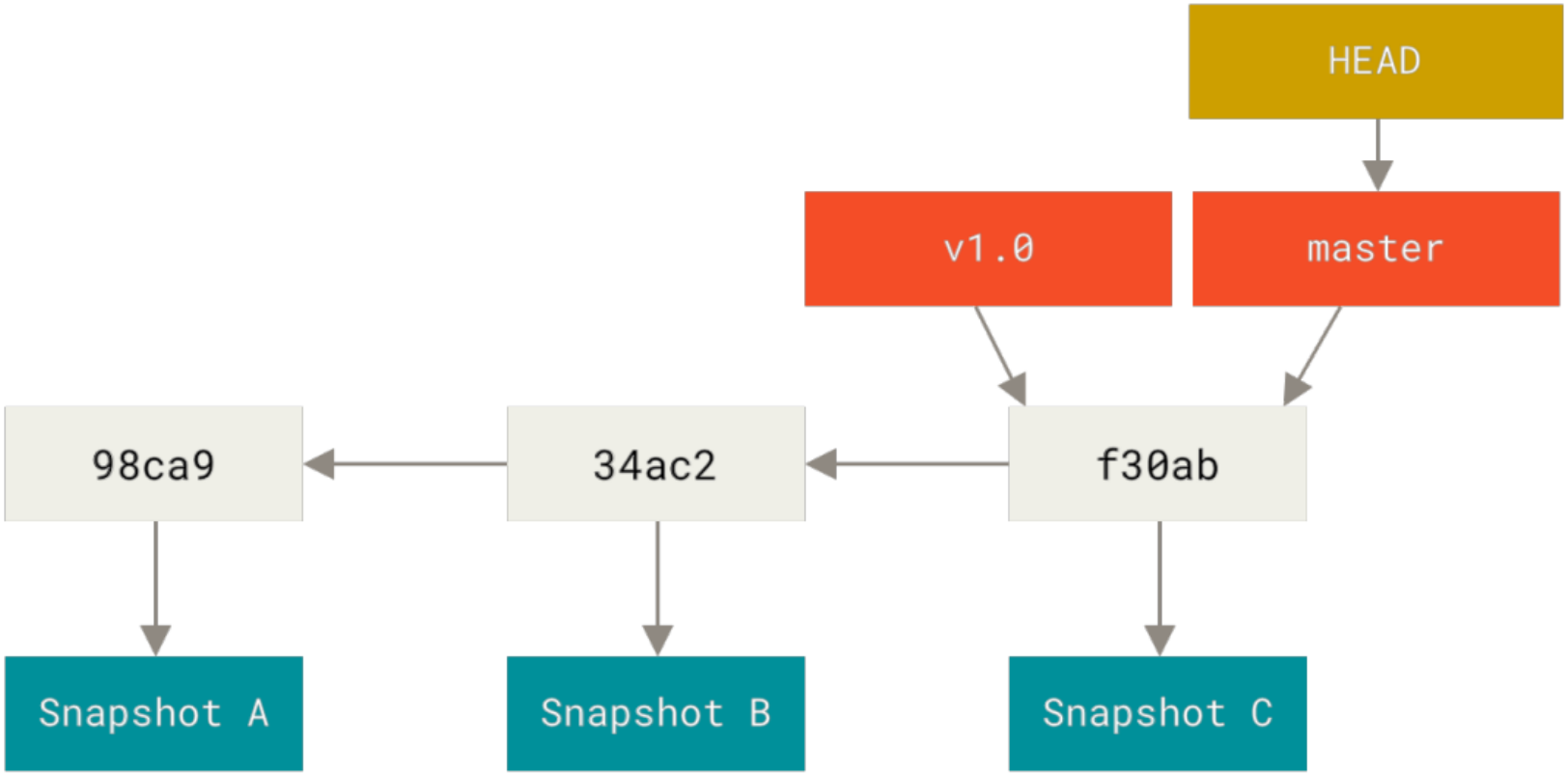
If you make some changes and commit again, the next commit stores a pointer to the commit that came immediately before it.



A branch and its commit history

- A branch in Git is simply a lightweight movable pointer to one of these commits. The default branch name in Git is master.
- As you start making commits, you're given a master branch that points to the last commit you made.
- Every time you commit, the master branch pointer moves forward automatically.
- The "master" branch in Git is not a special branch. It is exactly like any other branch. The only reason nearly every repository has one is that the git init command creates it by default and most people don't bother to change it.

A branch and its commit history

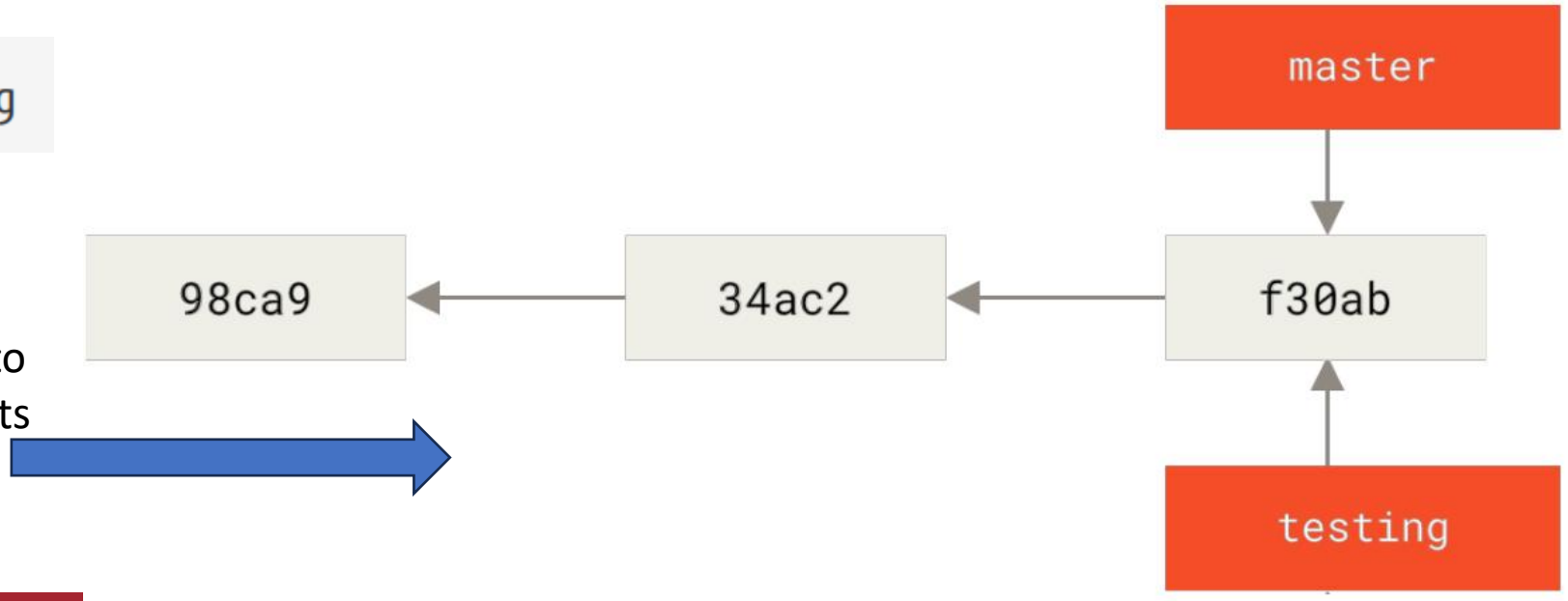


Creating a New Branch

- What happens when you create a new branch? Well, doing so creates a new pointer for you to move around. Let's say you want to create a new branch called testing. You do this with the git branch command:

```
$ git branch testing
```

Two branches pointing into the same series of commits



HEAD pointing to a branch

- In Git, this is a pointer to the local branch you're currently on. In this case, you're still on master. The git branch command only created a new branch — it didn't switch to that branch.



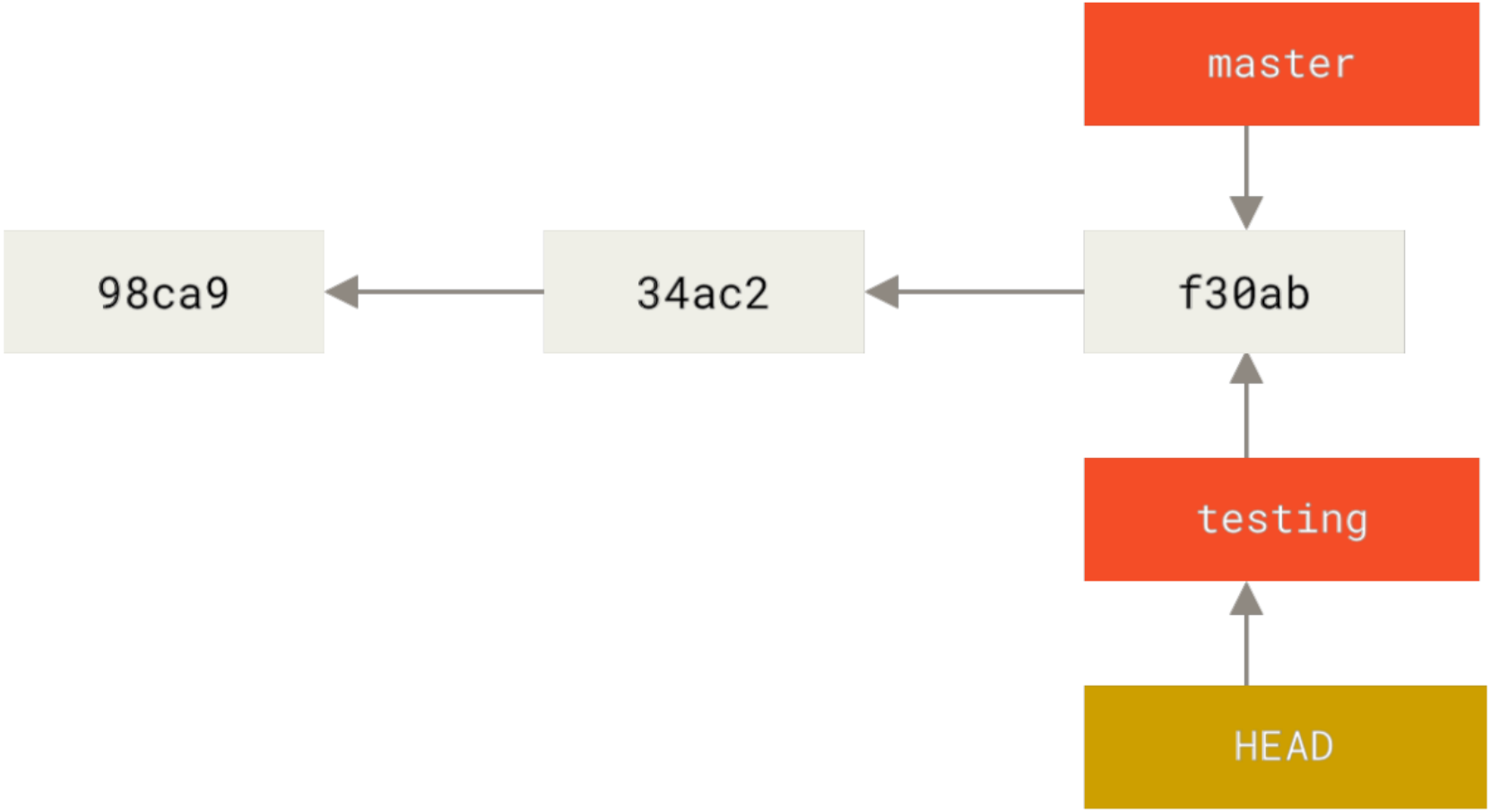
Switching Branches

- To switch to an existing branch, you run the git checkout command. Let's switch to the new testing branch:

```
$ git checkout testing
```

This moves HEAD to point to the testing branch.

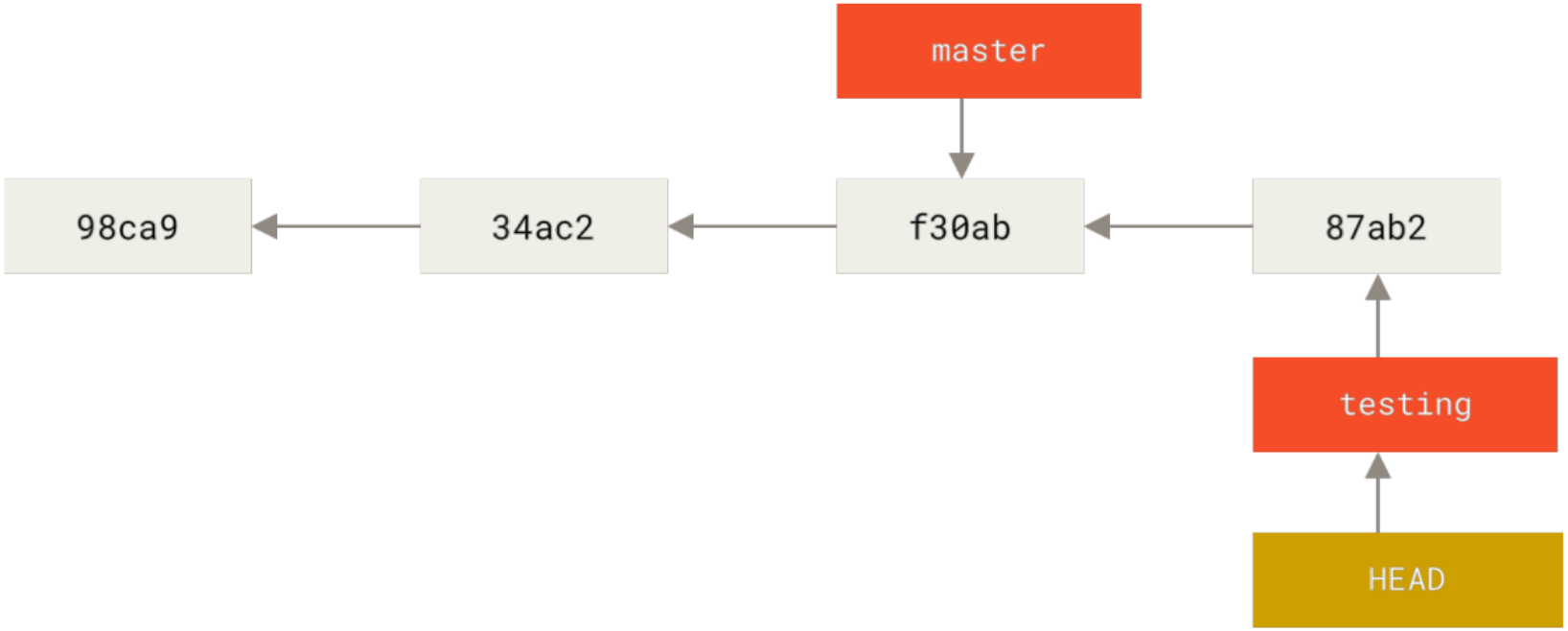
HEAD points to the current branch



The HEAD branch moves forward when a commit is made

- What is the significance of that? Well, let's do another commit:

```
$ vim test.rb
$ git commit -a -m 'Make a change'
```



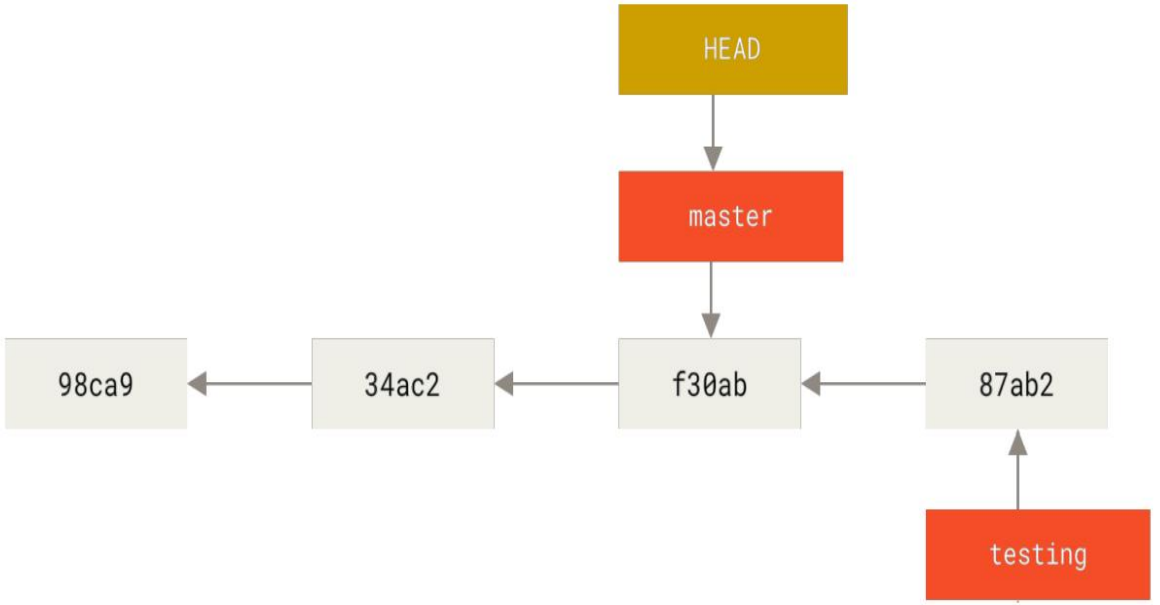
HEAD moves when you checkout

- This is interesting, because now your testing branch has moved forward, but your master branch still points to the commit you were on when you ran git checkout to switch branches. Let's switch back to the master branch:

```
$ git checkout master
```

That command did two things.

- It moved the HEAD pointer back to point to the master branch, and
- It reverted the files in your working directory back to the snapshot that master points to.



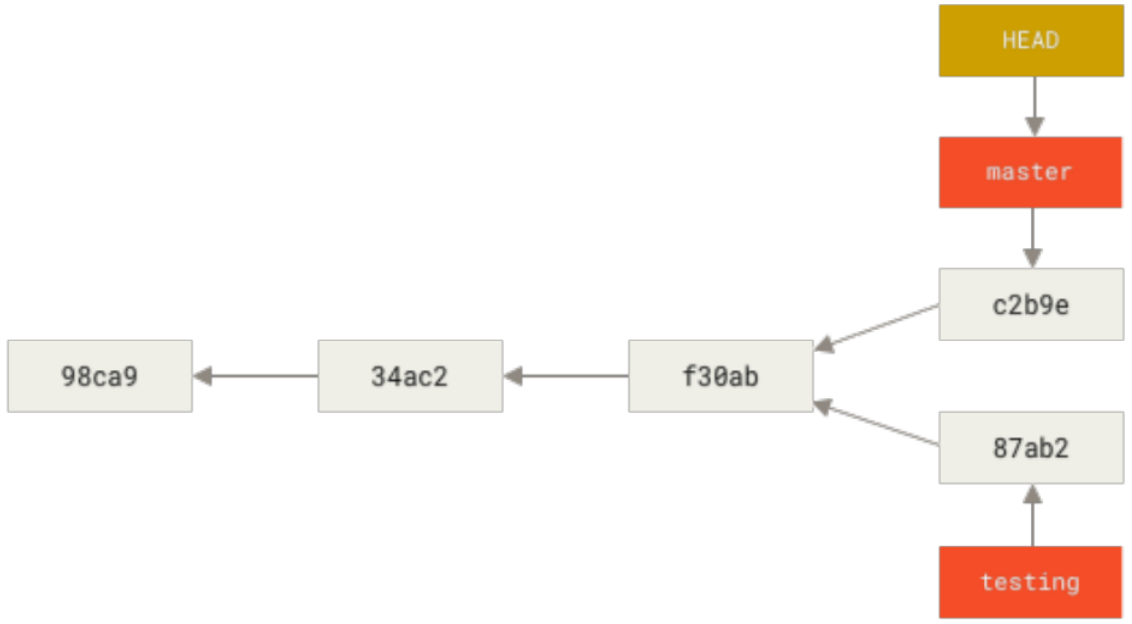
Divergent history

- Let's make a few changes and commit again:

```
$ vim test.rb
$ git commit -a -m 'Make other changes'
```

You created and switched to a branch, did some work on it, and then switched back to your main branch and did other work.

Both of those changes are isolated in separate branches: you can switch back and forth between the branches and merge them together when you're ready.



Why Git Branching?

- This is in sharp contrast to the way most older VCS tools branch, which involves copying all of the project's files into a second directory.
- This can take several seconds or even minutes, depending on the size of the project, whereas in Git the process is always instantaneous.
- Also, because we're recording the parents when we commit, finding a proper merge base for merging is automatically done for us and is generally very easy to do.
- These features help encourage developers to create and use branches often.

Example of the Branching on Git

Let's go through a simple example of branching and merging with a workflow that you might use in the real world. You'll follow these steps:

1. Do some work on a website.
2. Create a branch for a new user story you're working on.
3. Do some work in that branch.

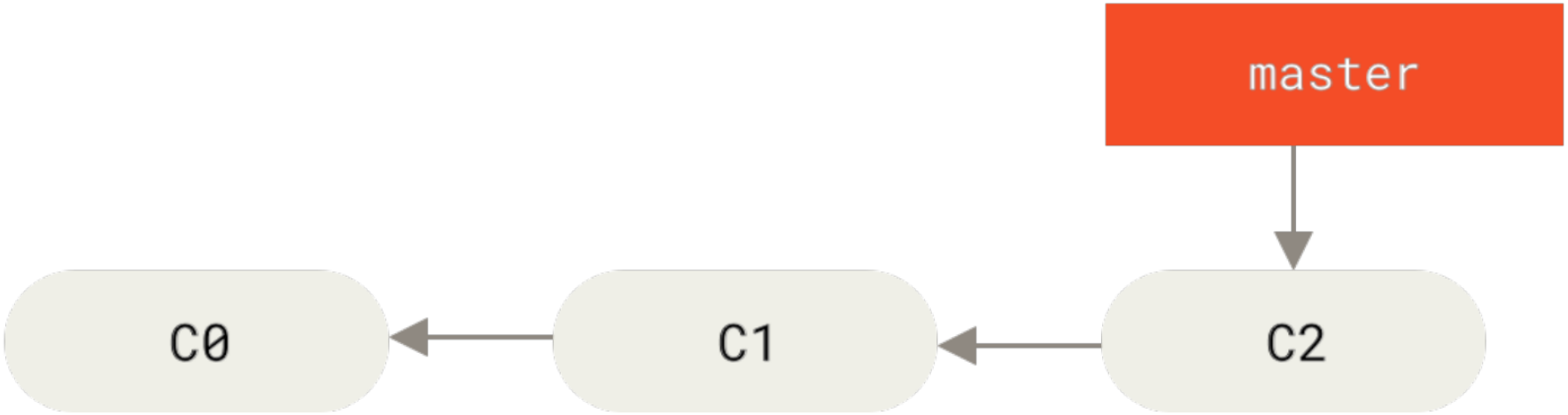
At this stage, you'll receive a call that another issue is critical and you need a hotfix. You'll do the following:

1. Switch to your production branch.
2. Create a branch to add the hotfix.
3. After it's tested, merge the hotfix branch, and push to production.
4. Switch back to your original user story and continue working.

Basic Branching and Merging

- Basic Branching

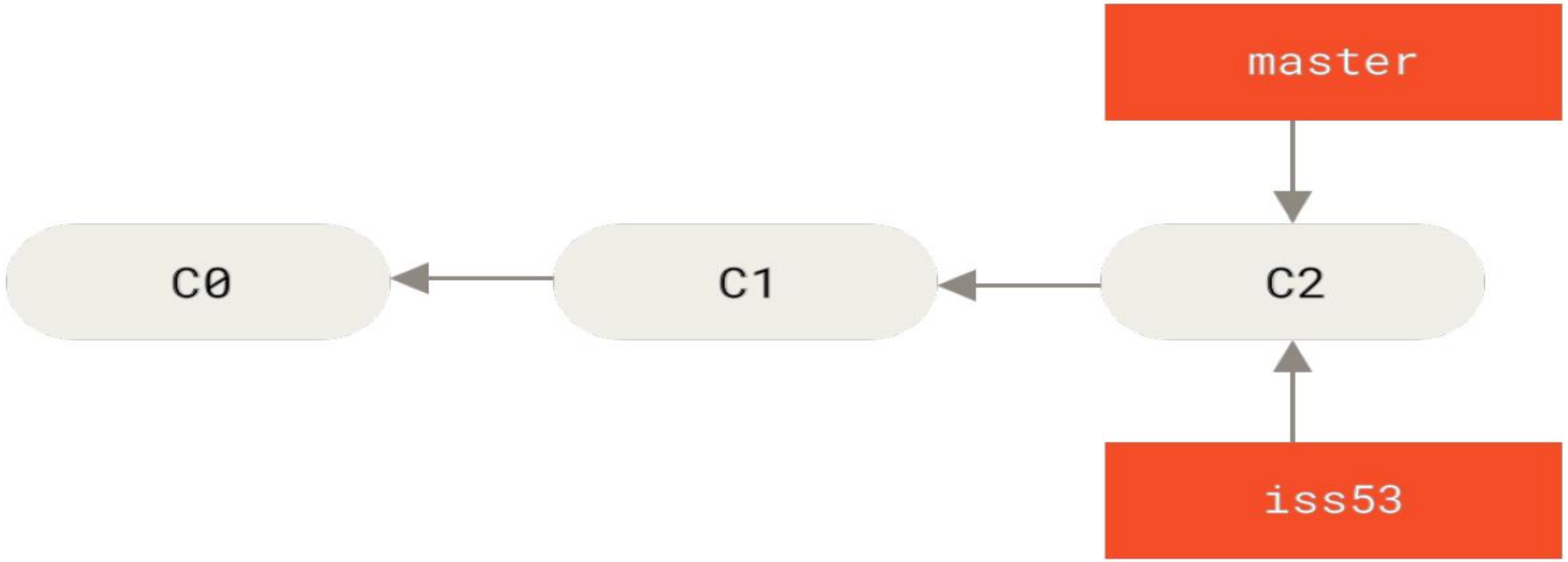
First, let's say you're working on your project and have a couple of commits already on the master branch.



Creating a new branch pointer

To create a new branch and switch to it at the same time, you can run the git checkout command with the -b switch:

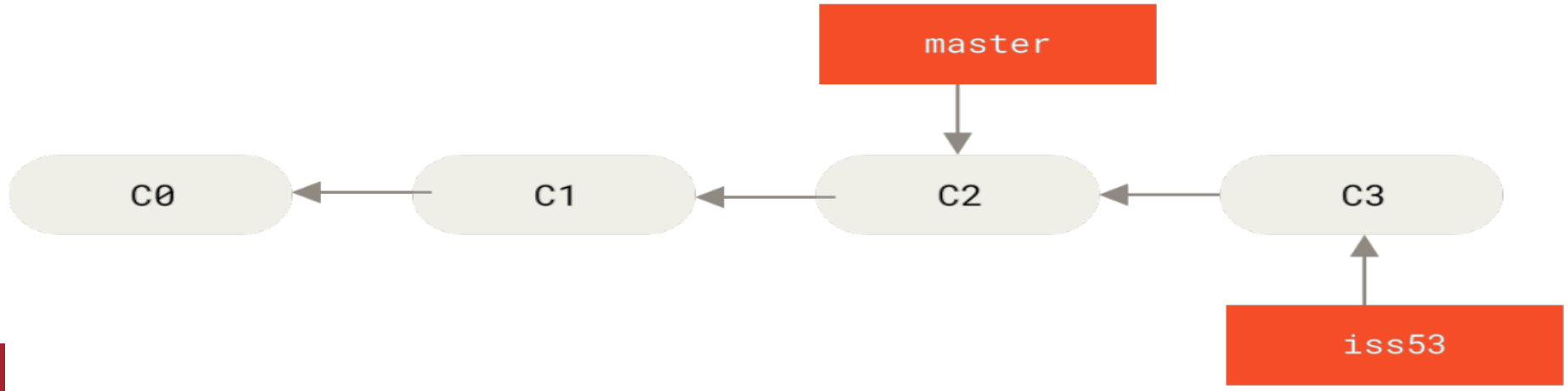
```
$ git checkout -b iss53
```



The iss53 branch has moved forward with your work

- You work on your website and do some commits. Doing so moves the iss53 branch forward, because you have it checked out (that is, your HEAD is pointing to it):

```
$ vim index.html
$ git commit -a -m 'Create new footer [issue 53]'
```



Hotfix branch based on master

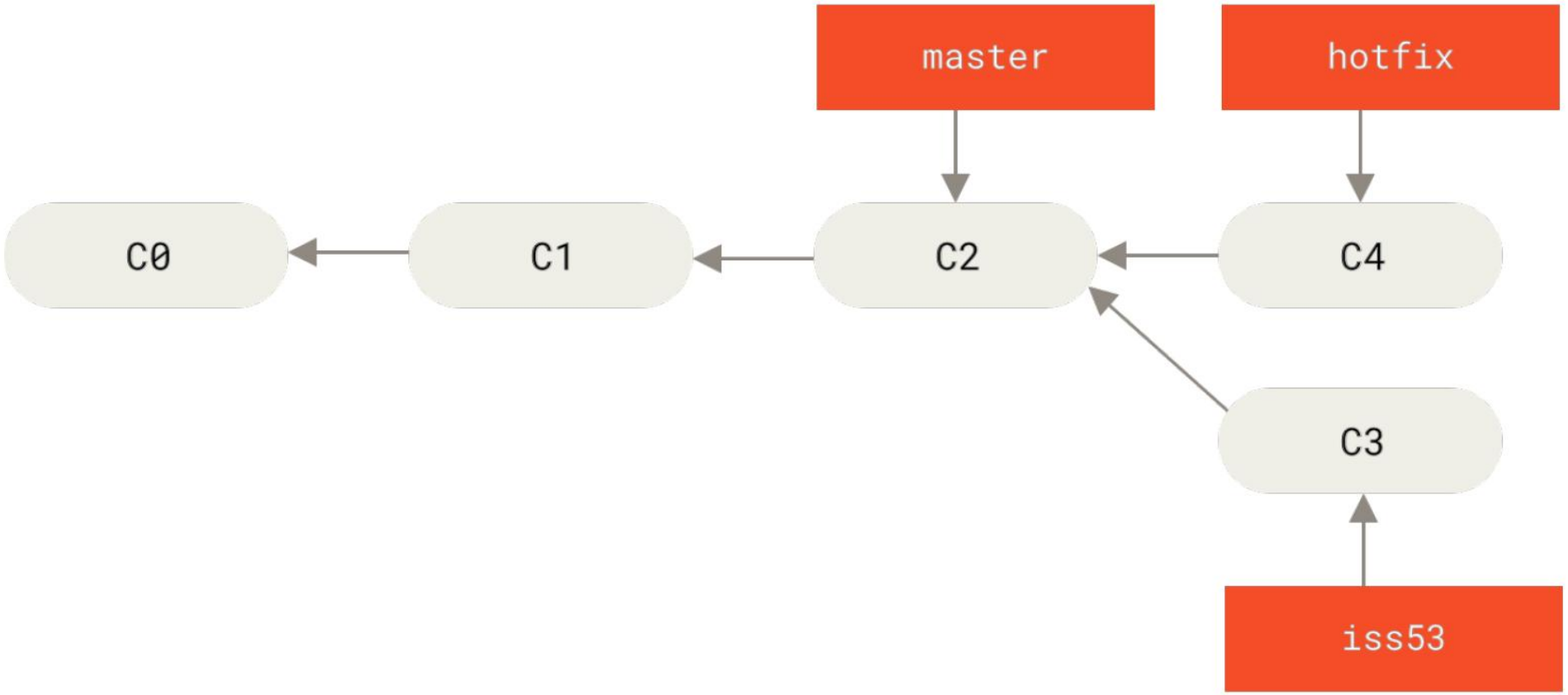
- Now you get the call that there is an issue with the website, and you need to fix it immediately. With Git, you don't have to deploy your fix along with the iss53 changes you've made, and you don't have to put a lot of effort into reverting those changes before you can work on applying your fix to what is in production. All you have to do is switch back to your master branch.

```
$ git checkout master
Switched to branch 'master'
```

Next, you have a hotfix to make. Let's create a hotfix branch on which to work until it's completed:

```
$ git checkout -b hotfix
Switched to a new branch 'hotfix'
$ vim index.html
$ git commit -a -m 'Fix broken email address'
[hotfix 1fb7853] Fix broken email address
 1 file changed, 2 insertions(+)
```

Hotfix branch based on master



master is fast-forwarded to hotfix

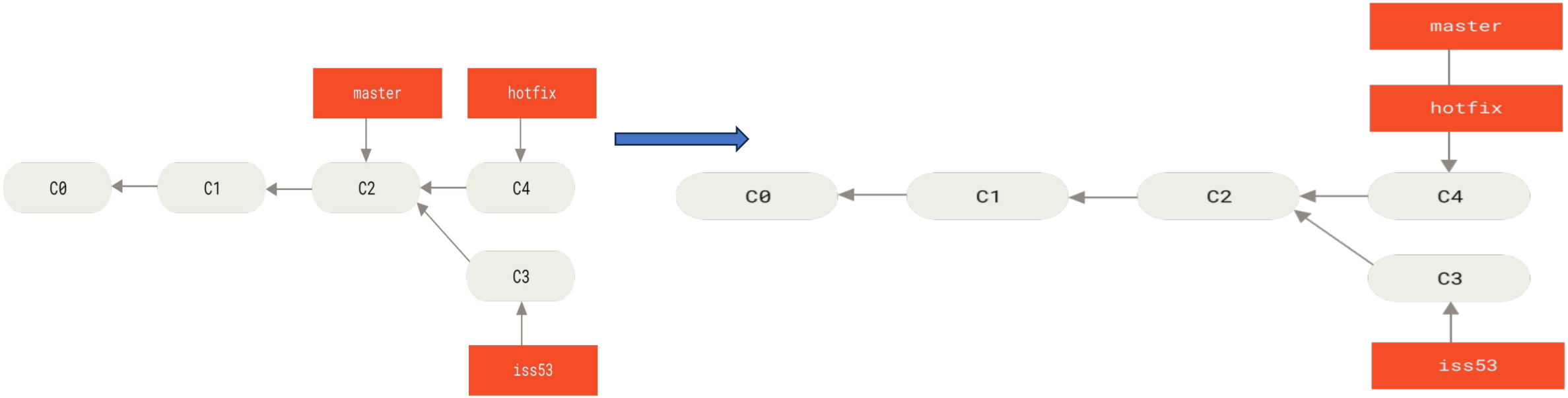
- You can run your tests, make sure the hotfix is what you want, and finally merge the hotfix branch back into your master branch to deploy to production. You do this with the git merge command:

```
$ git checkout master
$ git merge hotfix
Updating f42c576..3a0874c
Fast-forward
 index.html | 2 ++
 1 file changed, 2 insertions(+)
```

Fast forward merge

You'll notice the phrase "fast-forward" in that merge. Because the commit C4 pointed to by the branch hotfix you merged in was directly ahead of the commit C2 you're on, Git simply moves the pointer forward.

To phrase that another way, when you try to merge one commit with a commit that can be reached by following the first commit's history, Git simplifies things by moving the pointer forward because there is no divergent work to merge together — this is called a "fastforward."



Work continues on iss53

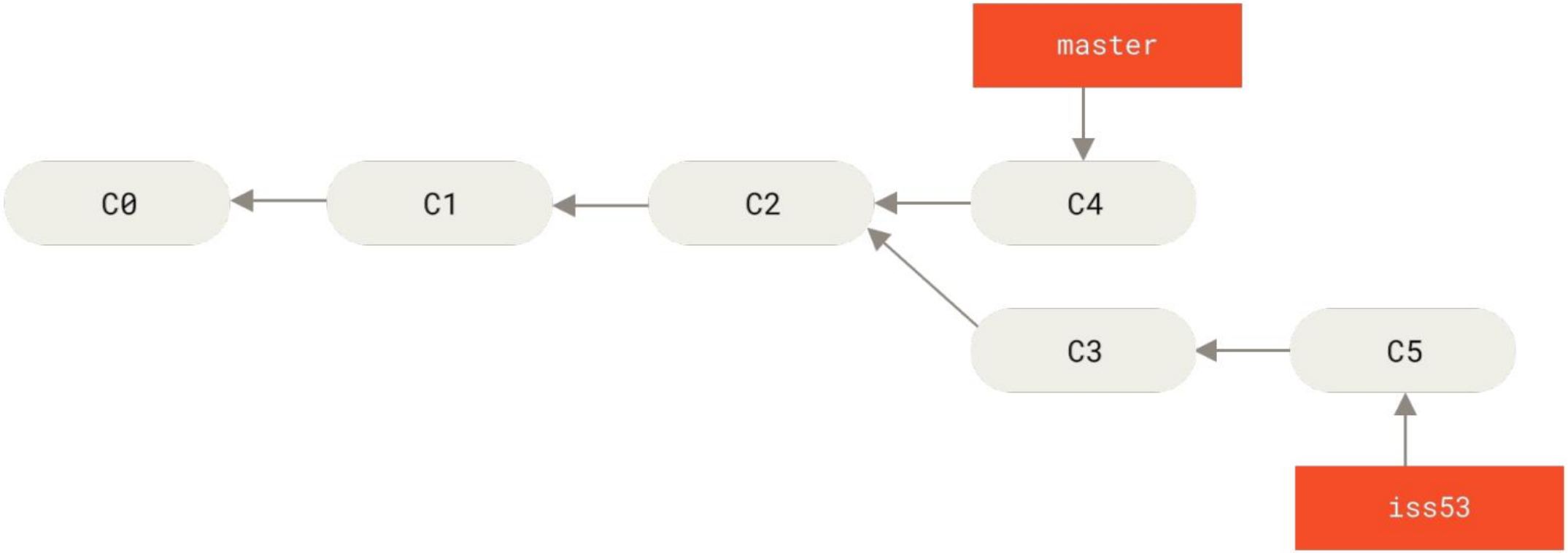
- After your super-important fix is deployed, you're ready to switch back to the work you were doing before you were interrupted. However, first you'll delete the hotfix branch, because you no longer need it — the master branch points at the same place. You can delete it with the -d option to git branch:

```
$ git branch -d hotfix
Deleted branch hotfix (3a0874c).
```

- Now you can switch back to your work-in-progress branch on issue #53 and continue working on

```
$ git checkout iss53
Switched to branch "iss53"
$ vim index.html
$ git commit -a -m 'Finish the new footer [issue 53]'
[iss53 ad82d7a] Finish the new footer [issue 53]
1 file changed, 1 insertion(+)
```

Work continues on iss53



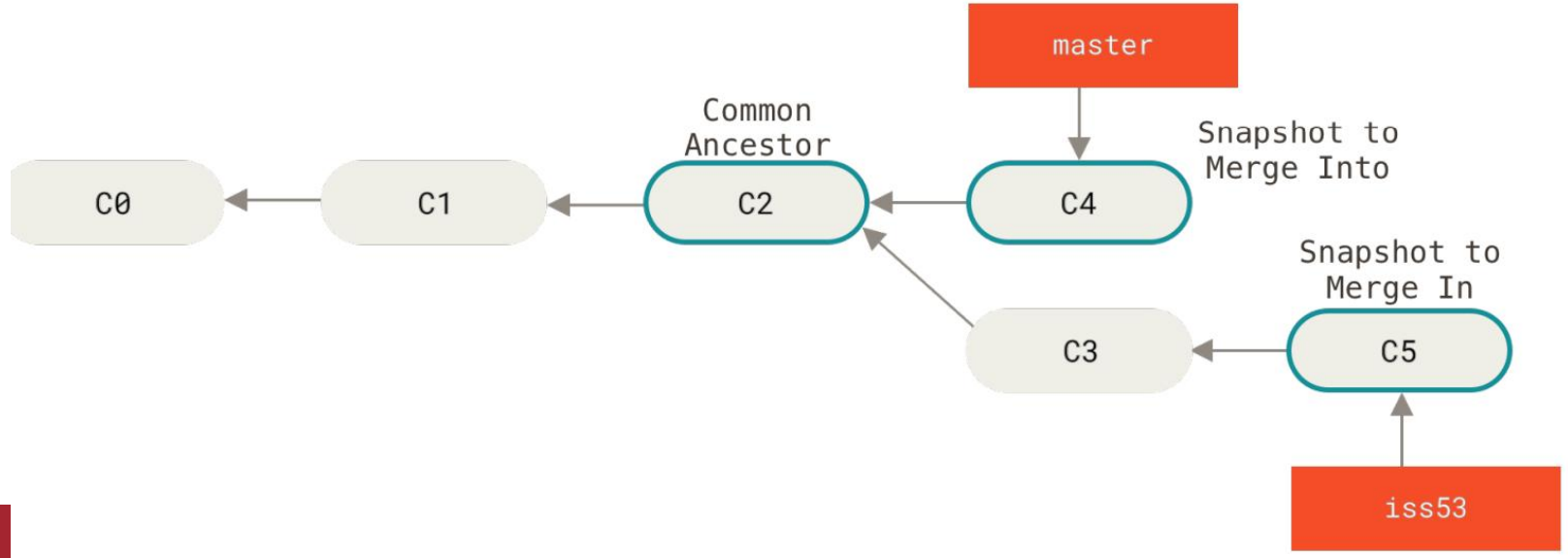
Basic Merging

- Suppose you've decided that your issue #53 work is complete and ready to be merged into your master branch. In order to do that, you'll merge your iss53 branch into master

```
$ git checkout master
Switched to branch 'master'
$ git merge iss53
Merge made by the 'recursive' strategy.
index.html |    1 +
1 file changed, 1 insertion(+)
```

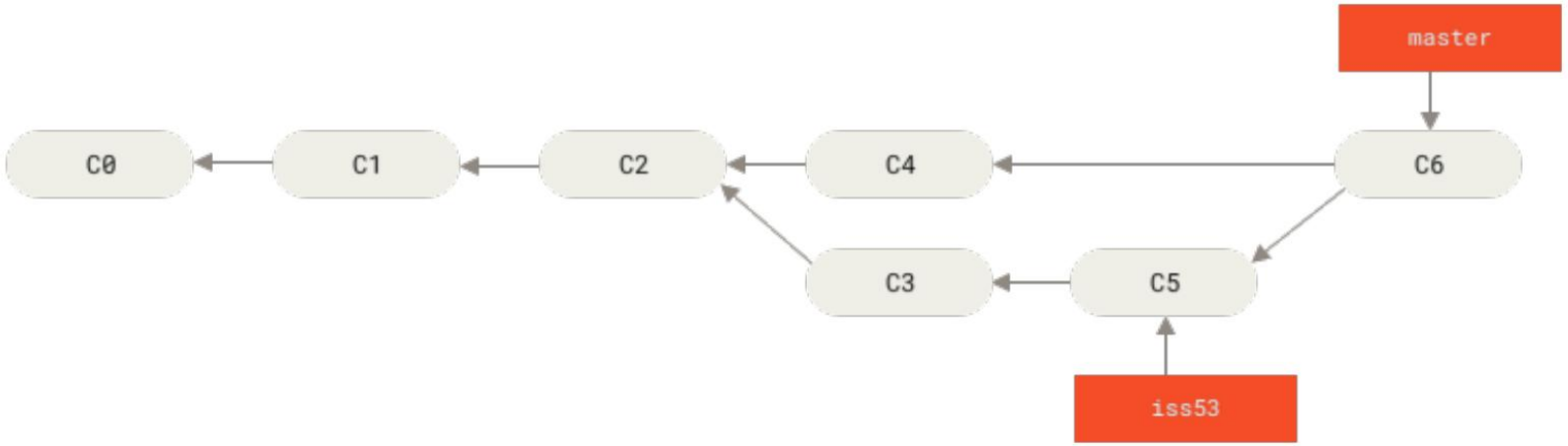
Three snapshots used in a typical merge

- This looks a bit different than the hotfix merge you did earlier. In this case, your development history has diverged from some older point. Because the commit on the branch you're on isn't a direct ancestor of the branch you're merging in, Git has to do some work. In this case, Git does a simple three-way merge, using the two snapshots pointed to by the branch tips and the common ancestor of the two.



A recursive merge commit

- Instead of just moving the branch pointer forward, Git creates a new snapshot that results from this three-way merge and automatically creates a new commit that points to it. This is referred to as a merge commit, and is special in that it has more than one parent.



Working with Remotes

What are Remote Repositories?

- Versions of your project hosted on the Internet or a network.
- Can be **read-only** or **read/write** for users.

Why Manage Remote Repositories?

- Essential for **collaborating** on Git projects.
- Enables **pushing** and **pulling** data to share work.

Key Remote Management Tasks:

- Adding and removing remote repositories.
- Managing remote branches.
- Defining tracked branches.

Showing Your Remotes

- To see which remote servers you have configured, you can run the git remote command. It lists the short names of each remote handle you've specified. If you've cloned your repository, you should at least see origin — that is the default name Git gives to the server you cloned from:

```
$ git clone https://github.com/schacon/ticgit
Cloning into 'ticgit'...
remote: Reusing existing pack: 1857, done.
remote: Total 1857 (delta 0), reused 0 (delta 0)
Receiving objects: 100% (1857/1857), 374.35 KiB | 268.00 KiB/s, done.
Resolving deltas: 100% (772/772), done.
Checking connectivity... done.
$ cd ticgit
$ git remote
origin
```

To clone a repo
Git clone <url>

Git will retrieve all the files associated with the repository and will copy them to your local machine

In addition to that git initializes a new repository on your machine giving you access to the fill history of the cloned project

```
LeelaNarasimha@QUAL-LTLEEL MINGW64 /d/projects/gitprojects/cloning
$ ls

LeelaNarasimha@QUAL-LTLEEL MINGW64 /d/projects/gitprojects/cloning
$ git status
fatal: not a git repository (or any of the parent directories): .git

LeelaNarasimha@QUAL-LTLEEL MINGW64 /d/projects/gitprojects/cloning
$ git clone https://github.com/JDStraughan/html5-snake.git
Cloning into 'html5-snake'...
remote: Enumerating objects: 46, done.
remote: Total 46 (delta 0), reused 0 (delta 0), pack-reused 46
Receiving objects: 100% (46/46), 9.39 KiB | 801.00 KiB/s, done.
Resolving deltas: 100% (15/15), done.

LeelaNarasimha@QUAL-LTLEEL MINGW64 /d/projects/gitprojects/cloning
$ ls
html5-snake/

LeelaNarasimha@QUAL-LTLEEL MINGW64 /d/projects/gitprojects/cloning
$ cd html5-snake/

LeelaNarasimha@QUAL-LTLEEL MINGW64 /d/projects/gitprojects/cloning/html5-snake (master)
$ ls
README.md  game.js  index.html  page.css

LeelaNarasimha@QUAL-LTLEEL MINGW64 /d/projects/gitprojects/cloning/html5-snake (master)
$
```

Any one can clone a repository from github, provided the repo is public.

You do not need to be owner or collaborator to clone the repo locally to your machine .

You just need the url from github.

Pushing up your changes to the github repo involves another process. You need permissions to do that

Remember that
Git clone is standard git command

It is not tied specifically to github. We can use it to clone repositories that are hosted anywhere.

Github with SSH

We can connect to github using two methods

1. Using HTTPS
2. Using SSH

When working with a GitHub repository, you'll often need to identify yourself to GitHub using your username and password.

An SSH key is an alternate way to identify yourself that doesn't require you to enter your username and password every time.

Using the SSH protocol, you can connect and authenticate to remote servers and services.

With SSH keys, you can connect to GitHub without supplying your username and personal access token at each visit.

To implement SSH keys in the system

We need to create the SSH key pair using

ssh-keygen -t ed25519 -C "your_email@example.com"

Next add your SSH key to the ssh agent

Ensure the ssh-agent is running using command

```
eval `ssh-agent -s`
```

Next add the ssh key to the ssh agent

```
ssh-add ~/.ssh/id_ed25519
```

We need to go to the github profile and add the ssh key pair in the settings.

We can check that we are authenticated to the github or not using

Ssh -T git@github.com

Generating a new SSH Key

Link

- https://docs.github.com/en/authentication/connecting-to-github-with-ssh/generating-a-new-ssh-key-and-adding-it-to-the-ssh-agent

```
LeelaNarasimha@QUAL-LTLEEL MINGW64 /d
$ ssh-keygen -t ed25519 -C "leelawebdev@gmail.com"
Generating public/private ed25519 key pair.
Enter file in which to save the key (/c/Users/leelanarasimha/.ssh/id_ed25519):
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /c/Users/leelanarasimha/.ssh/id_ed25519
Your public key has been saved in /c/Users/leelanarasimha/.ssh/id_ed25519.pub
The key fingerprint is:
SHA256:2vmXY3GgcS9k4VNbaMDhpDs7c6LVO3ltUg/SM+rRT3o leelawebdev@gmail.com
The key's randomart image is:
+--[ED25519 256]--+
|           .+o . |
|           +o + .|
|          ...+ o |
|          ..B..  |
|        S o*.+= .|
|       o ..+o=o*o|
|      . o * *== *|
|         + *=o BE|
|          . .o...|
+----[SHA256]-----+
```

SHH is running or not

```
LeelaNarasimha@QUAL-LTLEEL MINGW64 /d
$ eval `ssh-agent -s`
Agent pid 617
```

```
LeelaNarasimha@QUAL-LTLEEL MINGW64 /d
$ ssh-add ~/.ssh/id_ed25519
Identity added: /c/Users/leelanarasimha/.ssh/id_ed25519 (leelawebdev@gmail.com)
```

```
LeelaNarasimha@QUAL-LTLEEL MINGW64 /d
$ ssh-add ~/.ssh/id_ed25519
Identity added: /c/Users/leelanarasimha/.ssh/id_ed25519 (leelawebdev@gmail.com)

LeelaNarasimha@QUAL-LTLEEL MINGW64 /d
$ ssh -T git@github.com
The authenticity of host 'github.com (13.234.176.102)' can't be established.
RSA key fingerprint is SHA256:nThbg6kXUpJWGl7E1IGOCspRomTxdCARLviKw6E5SY8.
This key is not known by any other names
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added 'github.com' (RSA) to the list of known hosts.
Hi leelawebdev! You've successfully authenticated, but GitHub does not provide shell access.
```

Github Remote access

How do i get my code in github?

Option 1:
Existing Repo

If you already have an existing repo locally that you want to get on Github.

1. Create a new repo on github
2. Connect your local repo (add a remote)
3. Push up your changes to Github

How do i get my code in github?

Option 2:
Start from scratch

If you haven't begun work on the local repo.

1. Create new repo in github
2. Clone it down to your machine
3. Do some work locally
4. Push your changes to github

We need to make some how the local repo know about the github repo

We need to tell about our remote repository present on Github.

We need to setup the “destination” in the local repo.

Viewing Remotes

To view any existing remotes for your repository, we can run git remote or git remote -v

This command just displays a list of remotes. If you haven't added any remotes yet, you won't see anything.

```
LeelaNarasimha@QUAL-LTLEEL MINGW64 /d/projects/gitprojects/cloning/html5-snake (master)
$ git remote
origin

LeelaNarasimha@QUAL-LTLEEL MINGW64 /d/projects/gitprojects/cloning/html5-snake (master)
$ git remote -v
origin  https://github.com/JDStraughan/html5-snake.git (fetch)
origin  https://github.com/JDStraughan/html5-snake.git (push)

LeelaNarasimha@QUAL-LTLEEL MINGW64 /d/projects/gitprojects/cloning/html5-snake (master)
$ cd ..

LeelaNarasimha@QUAL-LTLEEL MINGW64 /d/projects/gitprojects/cloning
$ cd ..

LeelaNarasimha@QUAL-LTLEEL MINGW64 /d/projects/gitprojects
$ cd ecommerceProj

LeelaNarasimha@QUAL-LTLEEL MINGW64 /d/projects/gitprojects/ecommerceProj (master)
$ git remote -v

LeelaNarasimha@QUAL-LTLEEL MINGW64 /d/projects/gitprojects/ecommerceProj (master)
$
```

Adding Remote

Git remote add origin <url>

Origin is the short name for the url.

That means when ever i use the name origin, i am referring to the particular github url like an alias name

The name origin is the conventional git remote name, It is not at all special. It's just a name for a URL.

When we clone a Github repo, the default remote name setup for us is called origin. You can change Most people leave it as it is

How we have master as the default branch just like that origin is default, if needed we can change the name

Changing the name of the url from origin to user defined name

We can also use like this

Git remote add mygithuburl <url>

This means anytime you use the name "mygithuburl", we are referring to this particular Github repo url.

Commonly used remote name is origin by all of them.

Renaming the Remote

Renaming the remotes

Git remote rename <old-name> <new-name>

We can also remove the remote using
Git remote remove <name>

Git remote command

```
LeelaNarasimha@QUAL-LTLEEL MINGW64 /d/projects/gitprojects/ecommercePr
oj (master)
$ git remote

LeelaNarasimha@QUAL-LTLEEL MINGW64 /d/projects/gitprojects/ecommerceProj (master)
$ git remote -v

LeelaNarasimha@QUAL-LTLEEL MINGW64 /d/projects/gitprojects/ecommerceProj (master)
$ git remote add origin https://github.com/leelawebdev/sample-project.git

LeelaNarasimha@QUAL-LTLEEL MINGW64 /d/projects/gitprojects/ecommerceProj (master)
$ git remote
origin

LeelaNarasimha@QUAL-LTLEEL MINGW64 /d/projects/gitprojects/ecommerceProj (master)
$ git remote -v
origin  https://github.com/leelawebdev/sample-project.git (fetch)
origin  https://github.com/leelawebdev/sample-project.git (push)

LeelaNarasimha@QUAL-LTLEEL MINGW64 /d/projects/gitprojects/ecommerceProj (master)
$
```

```
LeelaNarasimha@QUAL-LTLEEL MINGW64 /d/projects/gitprojects/ecommerceProj (master)
$ git remote rename origin githuburl

LeelaNarasimha@QUAL-LTLEEL MINGW64 /d/projects/gitprojects/ecommerceProj (master)
$ git remote
githuburl
```

Removing the remote

```
LeelaNarasimha@QUAL-LTLEEL MINGW64 /d/projects/gitprojects/ecommerceProj (master)
$ git remote -v
githuburl       https://github.com/leelawebdev/sample-project.git (fetch)
githuburl       https://github.com/leelawebdev/sample-project.git (push)

LeelaNarasimha@QUAL-LTLEEL MINGW64 /d/projects/gitprojects/ecommerceProj (master)
$ git remote remove githuburl

LeelaNarasimha@QUAL-LTLEEL MINGW64 /d/projects/gitprojects/ecommerceProj (master)
$ git remote

LeelaNarasimha@QUAL-LTLEEL MINGW64 /d/projects/gitprojects/ecommerceProj (master)
$ git remote -v

LeelaNarasimha@QUAL-LTLEEL MINGW64 /d/projects/gitprojects/ecommerceProj (master)
$
```

Pushing git repo to remote repo

```
LeelaNarasimha@QUAL-LTLEEL MINGW64 /d/projects/gitprojects/ecommerceProj (master)
$ git push origin master
Enumerating objects: 24, done.
Counting objects: 100% (24/24), done.
Delta compression using up to 8 threads
Compressing objects: 100% (19/19), done.
Writing objects: 100% (24/24), 2.18 KiB | 279.00 KiB/s, done.
Total 24 (delta 6), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (6/6), done.
To https://github.com/leelawebdev/sample-project.git
 * [new branch]      master -> master

LeelaNarasimha@QUAL-LTLEEL MINGW64 /d/projects/gitprojects/ecommerceProj (master)
$ git branch
  dummy_branch
  feature/some_feature
  login_feature
* master
  new_branch
  new_feature
```

```
LeelaNarasimha@QUAL-LTLEEL MINGW64 /d/projects/gitprojects/ecommerceProj (new_branch)
$ ls
index.txt  newBranch.txt

LeelaNarasimha@QUAL-LTLEEL MINGW64 /d/projects/gitprojects/ecommerceProj (new_branch)
$ rm newBranch.txt

LeelaNarasimha@QUAL-LTLEEL MINGW64 /d/projects/gitprojects/ecommerceProj (new_branch)
$ rm index.txt

LeelaNarasimha@QUAL-LTLEEL MINGW64 /d/projects/gitprojects/ecommerceProj (new_branch)
$ ls
```

```
LeelaNarasimha@QUAL-LTLEEL MINGW64 /d/projects/gitprojects/ecommerceProj (new_branch)
$ git status
On branch new_branch
Changes not staged for commit:
  (use "git add/rm <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        deleted:    index.txt
        deleted:    newBranch.txt

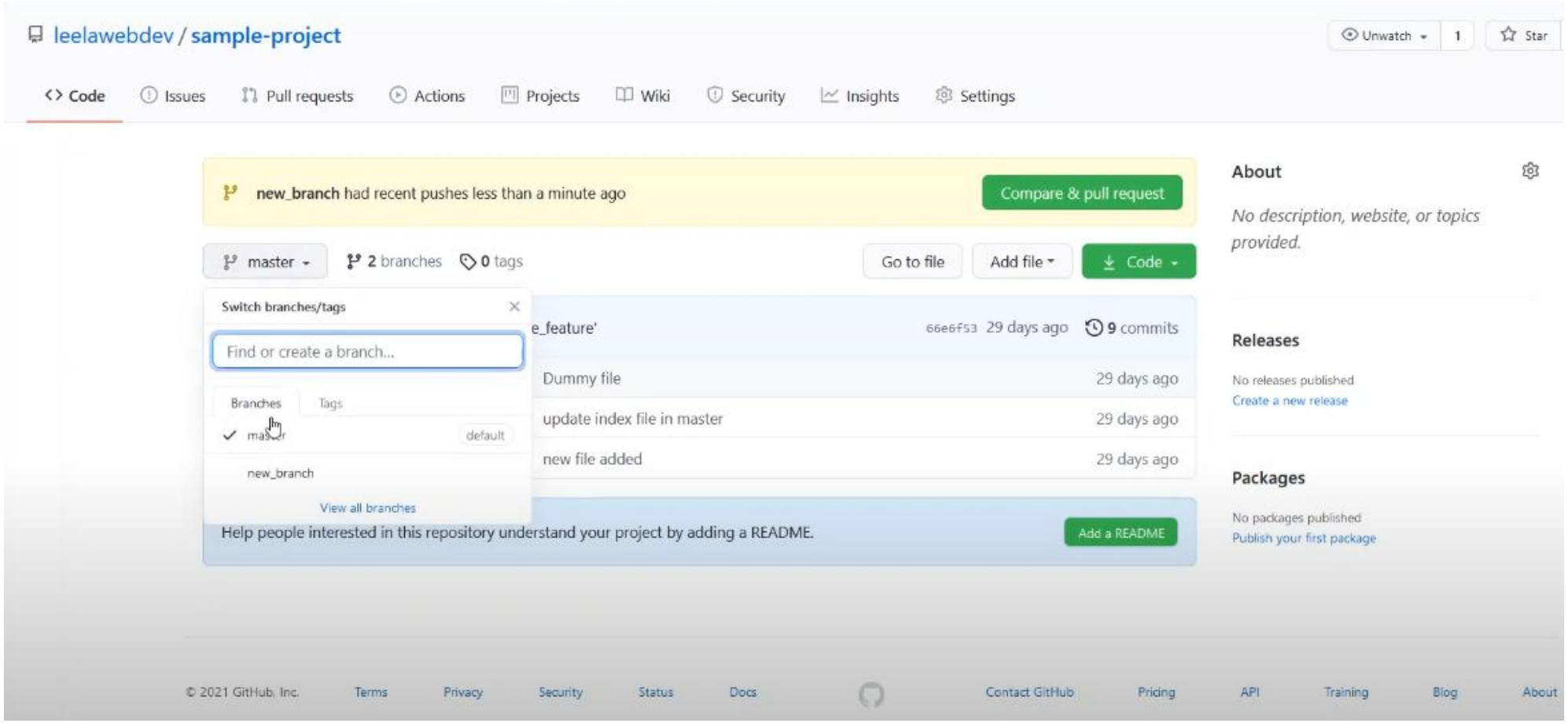
Untracked files:
  (use "git add <file>..." to include in what will be committed)
        leela.txt

no changes added to commit (use "git add" and/or "git commit -a")

LeelaNarasimha@QUAL-LTLEEL MINGW64 /d/projects/gitprojects/ecommerceProj (new_branch)
$ git add .

LeelaNarasimha@QUAL-LTLEEL MINGW64 /d/projects/gitprojects/ecommerceProj (new_branch)
$ git commit -m "delete and add file"
[new_branch 8b61f9e] delete and add file
 2 files changed, 1 deletion(-)
 delete mode 100644 index.txt
 rename newBranch.txt => leela.txt (100%)
```

```
LeelaNarasimha@QUAL-LTLEEL MINGW64 /d/projects/gitprojects/ecommerceProj (new_branch)
$ git push origin new_branch
Enumerating objects: 10, done.
Counting objects: 100% (10/10), done.
Delta compression using up to 8 threads
Compressing objects: 100% (5/5), done.
Writing objects: 100% (9/9), 850 bytes | 283.00 KiB/s, done.
Total 9 (delta 0), reused 0 (delta 0), pack-reused 0
remote:
remote: Create a pull request for 'new_branch' on GitHub by visiting:
remote:      https://github.com/leelawebdev/sample-project/pull/new/new_branch
remote:
To https://github.com/leelawebdev/sample-project.git
 * [new branch]      new_branch -> new_branch
```


leelawebdev / sample-project
Unwatch 1
Star
Code
Issues
Pull requests
Actions
Projects
Wiki
Security
Insights
Settings
new_branch had recent pushes less than a minute ago
Compare & pull request
master
2 branches
0 tags
Go to file
Add file
Code
Switch branches/tags
Find or create a branch...
Branches
Tags
master
default
new_branch
View all branches
e_feature'
66e6f53 29 days ago
9 commits
Dummy file
29 days ago
update index file in master
29 days ago
new file added
29 days ago
Help people interested in this repository understand your project by adding a README.
Add a README
About
No description, website, or topics provided.
Releases
No releases published
Create a new release
Packages
No packages published
Publish your first package
© 2021 GitHub, Inc.
Terms
Privacy
Security
Status
Docs
Contact GitHub
Pricing
API
Training
Blog
About